
DocOS: Towards Proactive Document-Guided Actions in GUI Agents

Jingjing Liu^{*1} Ziye Huang^{*1} Zihao Cheng^{*1} Zeming Liu¹ Jiahong Wu² Yuhang Guo³ Kehai Chen⁴
Yunhong Wang¹ Haifeng Wang⁵

Abstract

While Graphical User Interface (GUI) agents have shown promising performance in automated device interaction, they primarily depend on static parametric knowledge from pre-training or instruction tuning. This reliance fundamentally limits their ability to handle long-tailed tasks that require explicit procedural knowledge absent from model parameters, often forcing agents to resort to inefficient and brittle trial-and-error exploration. To mitigate this limitation, we introduce **Proactive Document-Guided Action** for GUI agents in dynamic, open-web environments, a novel paradigm that mirrors human problem-solving by enabling agents to autonomously search for relevant documentation to resolve long-tailed tasks. To evaluate agents' capability in this paradigm, we propose **DocOS**, a benchmark designed to assess document-guided problem solving in fully interactive environments. DocOS requires agents to autonomously navigate a web browser, locate relevant online documentation, comprehend procedural instructions, and faithfully ground them into executable GUI actions. Extensive experiments reveal that progress is strictly constrained by dual bottlenecks: agents struggle to reliably locate relevant information during proactive search and frequently fail to faithfully ground retrieved instructions into precise actions, pointing toward document-guided interaction as a crucial pathway for enabling self-evolving GUI agents in dynamic environments¹.

¹School of Computer Science and Engineering, Beihang University, Beijing, China ²School of Computer Science, Peking University, Beijing, China ³School of Computer Science and Technology, Beijing Institute of Technology, Beijing ⁴School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China ⁵Baidu Inc., Beijing, China. Correspondence to: Zeming Liu <zmlu@buaa.edu.cn>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

¹Dataset and codes are publicly available at <https://github.com/BUAA-IRIP-LLM/DocOS>

1. Introduction

Recent advances in Multimodal Large Language Models (MLLMs) have driven rapid progress in Graphical User Interface (GUI) agents, enabling automated interaction across a wide range of applications (Wang et al., 2024b; Nguyen et al., 2025; Yin et al., 2024; Lu et al., 2025; Liu et al., 2025), including web navigation (Yao et al., 2022; Deng et al., 2023; Zhou et al., 2024), desktop environments (Xie et al., 2024), and mobile devices (Zhang et al., 2023; Wang et al., 2024a). These developments mark an important step toward general agents capable of assisting humans with diverse real-world tasks.

Despite this progress, a fundamental limitation persists: existing GUI agents predominantly rely on *static parametric knowledge* acquired during pre-training or instruction tuning (Xu et al., 2025a; Liu et al., 2024; Zhou et al., 2025). While sufficient for common interactions, such knowledge is inherently incomplete and quickly becomes brittle when agents are confronted with long-tailed, application-specific tasks. These tasks often require explicit procedural instructions, precise option semantics, or rarely used functionalities that are absent from the model's fixed parameters. As illustrated in Figure 1, for a specific instruction like "Create a run configuration from a python template," the agent lacks the inherent knowledge to navigate the PyCharm interface directly. Instead, it must first navigate the open web to retrieve the official "Run/Debug Configurations" guide, utilizing this external document to ground its subsequent actions in the software. This behavior increases execution failures and hallucinations, fundamentally undermining the reliability and deployability of GUI agents in dynamic environments.

To mitigate this limitation, we introduce **Proactive Document-Guided Action**, a novel paradigm that mirrors human problem-solving by enabling agents to autonomously search the open web for documentation to resolve long-tailed tasks. Under this paradigm, agents are no longer restricted to static internal knowledge. Instead, they proactively navigate browsers to locate and comprehend documents in real time. This paradigm decouples an agent's reasoning capability from the storage of vast, domain-specific knowledge. By dynamically acquiring explicit procedural guidance on the fly, agents can faithfully ground their ac-

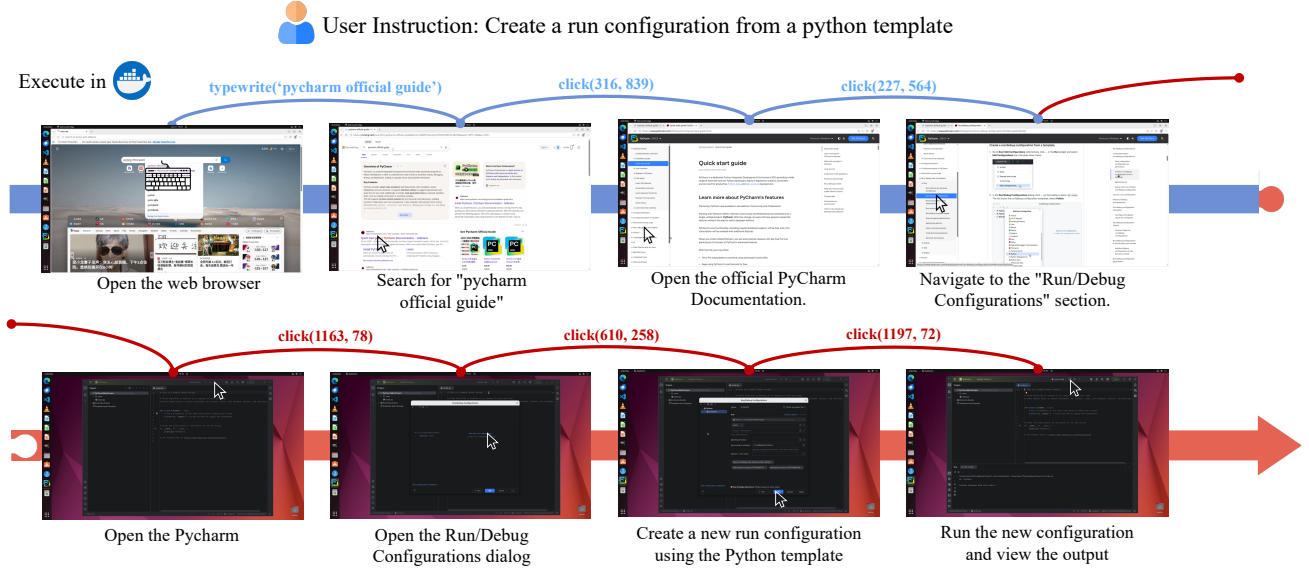


Figure 1. An example of a Proactive Document-Guided Action task. The workflow is divided into two distinct phases: **Proactive Knowledge Retrieval** (top row) and **Document-Grounded Execution** (bottom row).

tions in up-to-date documentation, enabling adaptation to long-tailed and evolving tasks without extensive retraining.

Guided by this paradigm, we propose **DocOS**, a benchmark designed to assess document-guided problem solving in fully interactive environments. Specifically, we curate a suite of high-difficulty instructions targeting long-tailed functionalities that are typically unsolvable without external documents. Crucially, we establish a dynamic execution environment (i.e., Docker) (Xie et al., 2024) where agents are not passively fed information and more details are in Appendix A. Instead, consistent with our paradigm, they are required to *autonomously navigate a web browser, locate relevant online documentation, comprehend procedural instructions, and faithfully ground them into executable GUI actions*. This setup moves beyond static, information-provided evaluations by explicitly assessing an agent’s end-to-end capability to autonomously search relevant web documentation and ground the acquired procedural documents into precise GUI actions.

To systematically evaluate current GUI agents under this paradigm, we conduct extensive experiments on DocOS. Specifically, we design two distinct task settings to decouple and assess the core capabilities **Proactively Document Search**, which evaluates the agent’s ability to locate documents via web browsing, and **Document-Grounded Execution**, which tests its capacity to implement instructions given the documentation. Results reveal that GUI agents are strictly constrained by dual bottlenecks: agents struggle to reliably locate relevant information during the proactive search phase, and even when provided with oracel document in the execution phase, they frequently fail to faithfully

ground retrieved instructions into precise actions. These findings highlight critical gaps in both document retrieval and document-conditioned execution, limiting end-to-end reliability in real-world deployment.

Overall, our contributions are summarized as follows:

- To the best of our knowledge, we are the first to propose **Proactive Document-Guided Action** for GUI agents, a novel paradigm that mirrors human problem-solving by enabling agents to autonomously search the open web for documents and apply them to solve long-tailed tasks.
- Guided by this paradigm, we introduce **DocOS**, a benchmark designed to evaluate GUI agents to autonomously navigate a web browser, locate relevant online documentation, comprehend procedural instructions, and accurately translate them into actions.
- We perform a systematic evaluation under two complementary settings: **Proactive Document Search** and **Document-Grounded Execution**. Our results show that the evaluated agents are fundamentally limited by **dual bottlenecks**: inaccurate information localization during document search and unfaithful grounding of instructions into actions, underscoring a critical gap toward robust, self-guided deployment.

2. Related Work

2.1. GUI Agents

Recent advances in MLLMs have enabled agents to perceive, reason about, and interact with GUI across diverse

Table 1. Comparison of virtual agent benchmarks. The columns indicate whether a benchmark supports proactive document search, document-grounded execution, interaction with dynamic web environments (i.e., non-static web pages), complex multi-step tasks, and fine-grained subtask-level evaluation.

	Proactively Document Search	Document-Grounded Execution	Dynamic Web Environment	Complex Task	Subtask Evaluation
OmniACT (Kapoor et al., 2024)	✗	✗	✗	✗	✗
ScreenSpot-Pro (Li et al., 2025)	✗	✗	✗	✗	✗
WebArena (Zhou et al., 2024)	✗	✗	✗	✗	✗
VisualWebArena (Koh et al., 2024)	✗	✗	✗	✗	✗
OSWorld (Xie et al., 2024)	✗	✗	✗	✗	✗
CRAB (Xu et al., 2025b)	✗	✗	✗	✗	✓
VideoGUI (Lin et al., 2024)	✗	✗	✗	✓	✓
GUI-World (Chen et al., 2024)	✗	✗	✗	✓	✗
AitW (Rawles et al., 2023)	✗	✗	✗	✓	✗
Mind2Web (Deng et al., 2023)	✗	✗	✗	✓	✗
Spider2-V (Cao et al., 2024)	✗	✗	✓	✓	✗
WebCanvas (Pan et al., 2024)	✗	✗	✓	✓	✓
OmniBench (Bu et al., 2025)	✗	✗	✓	✓	✓
DocOS (Ours)	✓	✓	✓	✓	✓

digital environments, rather than being limited to HTML or text based interfaces. Building on these foundations, a growing num of works focuses on specialized GUI agents designed for interactive environments, such as OS-Atlas (Wu et al., 2024b), DeepMiner-Mano (Fu et al., 2025), OpenCUA (Wang et al., 2025), CogAgent (Hong et al., 2024), OS-Copilot (Wu et al., 2024a) and Mobile-Agent (Ye et al., 2025) leveraging high-resolution visual perception to locate and interact with UI elements directly. However, most existing GUI agents implicitly rely on static parametric knowledge, and encounter great challenge when require knowledge beyond the model’s training distribution. To release this issue, several approaches (Xie et al., 2025; Zhang et al., 2025) introduce retrieval-augmented mechanisms, allowing agents to access additional information at inference time. But these methods typically assume that relevant documents are pre-collected, and do not capture the challenge of autonomous knowledge acquisition in open environments.

2.2. Benchmarks for GUI Agent

Current GUI benchmarks in Table 1 proposed for GUI Agents primarily focus on evaluating their ability to accomplish GUI-based tasks under different interaction settings. Recent benchmarks also evaluate broader autonomous agent capabilities, including proactivate conversation (Liu et al., 2020; Shi et al., 2023; Liu et al., 2021; 2022), translation (Li et al., 2020; Chen et al., 2025), and tool-use (Liang et al., 2025; Li et al., 2026), but overlook proactive document-guided GUI interaction. Static benchmarks like Mind2Web (Gou et al., 2025), OmniACT (Kapoor et al., 2024), and ScreenSpot-Pro (Jurmu et al., 2008) are constructed from offline data and assess the agent’s ability to predict actions or trajectories without real-time environment feedback, which do not capture the challenges of interac-

tive execution. In contrast, dynamic benchmarks including OSWorld (Xie et al., 2024), WebArena (Zhou et al., 2024) provide realistic, fully interactive environments that allow agents to execute actions, observe real-time feedback, and adapt their strategies accordingly. However, they typically assume that agents already possess the knowledge required to complete tasks and remain a gap of self-guided deployment. While Spider2-V (Cao et al., 2024), WebCanvas (Pan et al., 2024), and OmniBench (Bu et al., 2025) build dynamic online environment to evaluate web navigation tasks, they are limited to information retrieval and do not assess document-grounded execution. To the best of our knowledge, no existing benchmark systematically evaluates the end-to-end capability of GUI agents to actively search for updating frequently online documentation and ground retrieved procedural knowledge into executable GUI actions within a unified interactive environment. To alleviate this problem, we introduce DocOS, which evaluating GUI agents in Autonomous Document Search and Document-Grounded Execution.

3. DocOS

To evaluate the ability of GUI agents to retrieval in dynamic web environment and execute following the retrieval results, we construct DocOS that consists 817 high-quality desktop tasks across 20 applications to evaluate the ability of agents to search, retrieve, understand external online resources and execute complex, multi-step actions in realistic desktop environments.

3.1. Task Formulaton

We formalize the interaction between the GUI agent and the digital environment as a Partially Observable Markov

Decision Process (POMDP), defined by the tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \mathcal{I}, \mathcal{G}, \mathcal{V} \rangle$. Here, $\mathcal{S}$ represents the latent states of the operating system and applications, $\mathcal{A}$ denotes the space of permissible actions (e.g., clicking, typing, scrolling), and $\mathcal{O}$ is the observation space consisting of visual screenshots and accessibility trees. $\mathcal{T} : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ represents the transition dynamics of the environment, $\mathcal{I}$ is the natural language user instruction, $\mathcal{G}$ is the goal conditions which defining the expected state for task completion, and $\mathcal{V} : \mathcal{S} \times \mathcal{G} \rightarrow \{0, 1\}$ represents a verifier that determines whether the current state satisfies the goal.

At each time step t , the agent receives an observation $o_t \in \mathcal{O}$ derived from the current state s_t and executes an action $a_t \in \mathcal{A}$ based on its policy π . The environment then transitions to a new state s_{t+1} according to $\mathcal{T}$.

In the context of the **Proactively Document-Guided Action** task, the agent cannot rely solely on internal parametric knowledge to solve $\mathcal{I}$ due to the long-tailed or domain-specific nature of the task. Instead, the process is decomposed into two distinct sequential phases: *Proactive Knowledge Retrieval* and *Document-Grounded Execution*.

Proactive Knowledge Retrieval In this initial phase, the agent’s objective is not to immediately fulfill the user instruction $\mathcal{I}$, but to actively acquire external information required to solve it. The agent interacts with a web browser environment to search for a relevant manual or documentation. The policy for this phase, denoted as π_{retr} , generates actions to navigate search engines and help centers:

$$a_t \sim \pi_{\text{retr}}(a_t \mid o_t, h_{t-1}, \mathcal{I}), \quad \text{for } 0 \leq t < T_{\text{retr}} \quad (1)$$

where h_{t-1} represents the interaction history. This phase concludes at time step T_{retr} when the agent identifies and retrieves a target document $\mathcal{D}$, which contains the necessary procedural knowledge or semantic explanations.

Document-Grounded Execution Upon retrieving the document $\mathcal{D}$, the agent transitions to the execution phase. The goal shifts to completing the original user instruction $\mathcal{I}$ by grounding the actions in the retrieved context $\mathcal{D}$. The policy π_{exec} operates as follows:

$$a_t \sim \pi_{\text{exec}}(a_t \mid o_t, h_{t-1}, \mathcal{I}, \mathcal{D}), \quad \text{for } t \geq T_{\text{retr}} \quad (2)$$

Unlike standard GUI agents that depend solely on $\mathcal{I}$, our formulation explicitly conditions the action generation on the unstructured knowledge $\mathcal{D}$. The task is considered successful if and only if the final state s_{End} satisfies the goal condition specified by $\mathcal{I}$, verifying that the agent has correctly interpreted and applied the external knowledge.

3.2. Data Collection

To construct a scalable and semantically meaningful task set for GUI agents, the data collection pipeline consists of three

stages as shown in Figure 2, including task construction, document collection, and task filtering. This pipeline leverages authoritative documentation as the grounding source and combines task construction with careful filtering.

Task Construction We manually construct task instructions along with their corresponding prerequisite to ensure high quality and realism. The tasks are designed to be realistic and goal-oriented, closely reflecting how users would interact with the application in practice. Following current works (Cao et al., 2024; Koh et al., 2024), we categorize tasks into three difficulty levels based on the number of required execution steps: Easy tasks can be completed within four steps or fewer, typically involving direct and atomic interactions; Middle tasks require five to seven steps, often involving multiple interface components or conditional actions; Hard tasks consist of eight or more steps, representing complex workflows that require sustained planning and multi-stage interaction.

Document Collection We collect authoritative official documentation from a diverse set of real-world applications. These documents are sourced from publicly available official websites, which provide reliable and comprehensive descriptions of application functionalities, user workflows, and interface behaviors. To ensure scalability and consistency across applications, we design an automated crawling and preprocessing script that automatically retrieves and cleans the raw web pages for each application. The script removes non-informative elements such as navigation bars, advertisements, and redundant boilerplate content, while preserving semantically meaningful components including functional descriptions, step-by-step instructions, and UI-related explanations. The resulting cleaned documents serve as structured and reliable reference for Proactive knowledge retrieval.

Task Filtering The tasks suffer from issues such as semantic ambiguity, and infeasible execution. To alleviate these problems, we introduce the task filtering stage that removes the low-quality tasks. Specifically, we assess whether each task is (1) semantically consistent with the source documentation, (2) clearly specified and unambiguous, and (3) realistically executable in a real application environment. As a result, we select 817 high-quality samples from thousands of candidates.

3.3. Quality Control and Validation

We adopt a multi-stage quality control pipeline that combines cross-verification, validation of external resources to improve task executability, clarity and evaluation stability. Annotators execute the task following the provided instructions and external instructional resources within the target

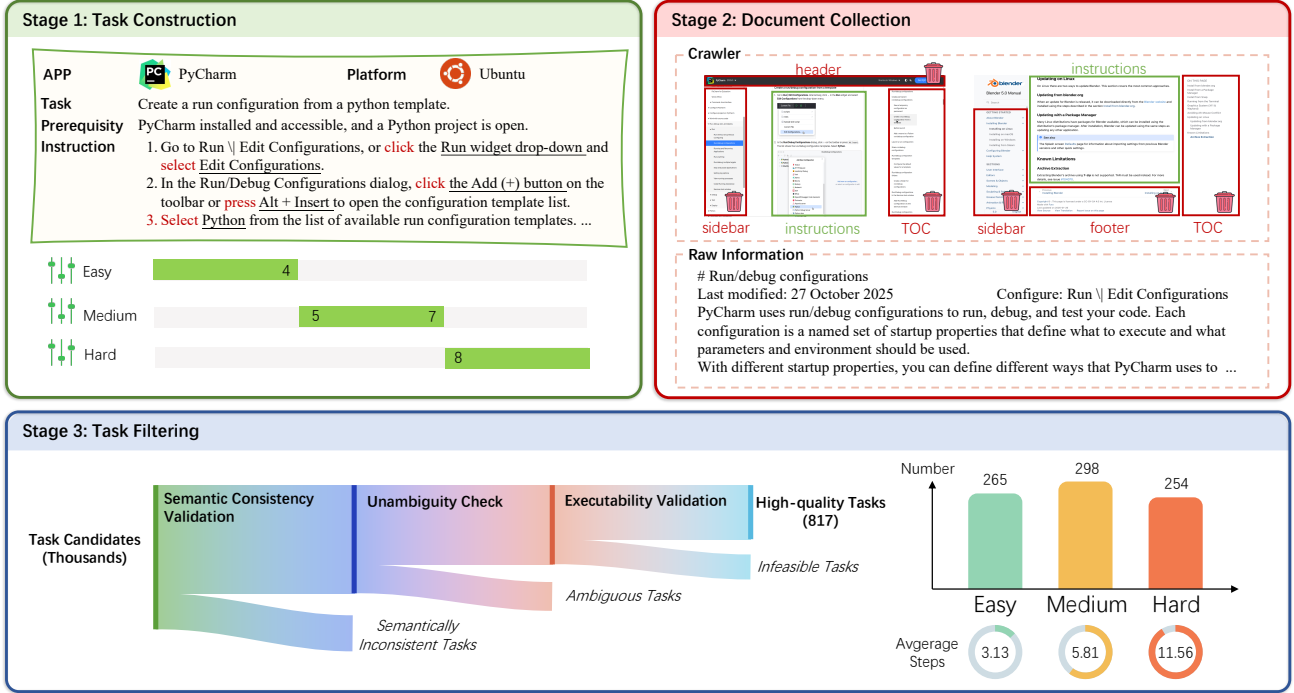


Figure 2. The data construction pipeline of DocOS consists of 3 stages. **Stage 1: Task Construction** involves defining task instructions, prerequisites, and difficulty levels (Easy, Medium, Hard) based on execution steps. **Stage 2: Document Collection** utilizes an automated crawler to retrieve and parse official documentation, extracting structured raw information (e.g., instructions, headers). **Stage 3: Task Filtering** implements a rigorous quality control funnel—validating semantic consistency, unambiguity, and executability—to select 817 high-quality tasks from thousands of candidates.

software environment, and report whether the task can be completed, as well as any ambiguities, or failures encountered during execution. To verify external resource validity stability, and relevance to the task objective, we perform both automated accessibility checks and manual inspection. Resources that are inaccessible, unstable, or weakly aligned with the task goal are removed to prevent failures unrelated to agent performance. This preprocessing step is only used to construct the ground-truth document; the agent evaluation environment remains fully dynamic and unfiltered. Following previous work (Xie et al., 2024), We sample approximately 20% (160 tasks) of the data to conduct human evaluations for data quality and find that 92.50% (148 tasks) of the data satisfies the requirements of semantic consistency, unambiguous instructions, and executability.

3.4. Data Statistics

As illustrated in Table 2, DocOS comprises 817 task instances spanning 20 applications, which are divide into three difficulty levels: Easy, Medium, and Hard. The number of tasks is relatively balanced among the three levels, with 265 easy tasks, 298 medium tasks, and 254 hard tasks. More statistic details and data examples in DocOS can be found in Appendix B.

Table 2. Statistic of DocOS.

Statistic	Level			Total
	Easy	Medium	Hard	
# APP	15	17	19	20
# Tasks	265	298	254	817
# Avg. Steps	3.13	5.81	11.56	6.72
# Avg. Prompt Len.	886.54	1286.69	2204.13	1442.12

4. Experiments

4.1. Experiment Setting

Baselines Following prior work (Bu et al., 2025), we conduct a comprehensive experiment on DocOS using Qwen3-VL-8B (Bai et al., 2025), UI-TARS-1.5-7B (Qin et al., 2025), MAI-UI-8B (Zhou et al., 2025), GELab-Zero-4B-preview (Yan et al., 2025), Aguis-7B (Xu et al.), and GUI-Owl-7B (Ye et al., 2025).

Metrics To comprehensively evaluate the capabilities of DocOS, we divide the evaluation process into two distinct phases: (1) **Documentation Search**, where the agent must navigate to the correct target page to retrieve necessary information, and (2) **Task Execution**, wherein the agent must accomplish the specific user objective based on the

Table 3. Main results on Easy, Medium, and Hard subsets.

Model	Easy			Medium			Hard			Avg.		
	TUI↑	HPP↑	TCR↑	TUI↑	HPP↑	TCR↑	TUI↑	HPP↑	TCR↑	TUI↑	HPP↑	TCR↑
GELab-Zero-4B-preview	5.88	51.47	5.17	0.00	47.83	6.63	5.88	62.25	2.12	3.23	52.78	4.36
UI-TARS-1.5-7B	4.26	56.12	30.86	1.91	54.41	17.32	1.20	56.08	13.08	2.48	55.48	17.31
MAI-UI-8B	3.00	47.47	20.99	2.62	45.66	7.82	3.10	42.83	10.00	2.89	45.34	10.96
Aguvis-7B	0.00	0.00	5.97	0.00	0.00	1.23	0.00	0.00	1.29	0.00	0.00	1.95
GUI-Owl-7B	0.00	13.97	9.88	0.00	16.79	6.15	5.00	21.67	3.85	1.54	17.33	5.58
Qwen3-VL-8B	44.53	51.07	7.41	39.60	55.89	3.35	42.91	58.82	3.85	42.23	55.21	4.23

retrieved context. Accordingly, we design specific metrics for each phase to ensure a granular assessment.

Phase 1: Documentation Search Metrics This phase assesses the agent’s ability to perceive visual cues and execute correct hierarchical navigation sequences. Since the agent operates on raw pixels, we utilize URL-based metrics as an objective proxy for navigation precision.

Target URL Inclusion (TUI) TUI measures whether the agent successfully locates the target documentation. **Justification:** In dynamic web environments, servers often append variable parameters (e.g., session IDs, tracking tokens) to URLs, making exact string matching overly rigid. Therefore, we adopt an inclusion-based criterion: a search is considered successful if the ground truth URL is a substring of the agent’s final retrieved URL.

$$TUI = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(U_{gt}^{(i)} \subseteq U_{retrieved}^{(i)}) \quad (3)$$

where N is the total number of tasks, U_{gt} and $U_{retrieved}$ denote the ground truth and retrieved URLs respectively, and $\mathbb{I}(\cdot)$ is the indicator function.

Hierarchical Path Progress (HPP) HPP evaluates the "depth" of the correct navigation path achieved by the agent. We treat a URL as a sequence of directory segments $\{s_1, s_2, \dots, s_k\}$ delimited by forward slashes ('/'). HPP is the ratio of the matching prefix length to the total GT path length:

$$HPP = \frac{1}{N} \sum_{i=1}^N \frac{|\text{LCP}(S_{gt}^{(i)}, S_{retrieved}^{(i)})|}{|S_{gt}^{(i)}|} \quad (4)$$

where $\text{LCP}(\cdot)$ denotes the Longest Common Prefix. **Justification:** Binary success metrics (like TUI) fail to capture partial progress. HPP allows us to distinguish between agents that are "completely lost" and those that navigate correctly through the site hierarchy but fail at the final leaf node, providing deeper insight into the agent’s reasoning logic.

Phase 2: Task Execution Metrics This phase evaluates the end-to-end utility of the agent, focusing on whether the user’s ultimate intent was satisfied.

Task Completion Rate (TCR) TCR quantifies the overall success rate of the system. It is defined as the ratio of successfully completed tasks to the total number of evaluation tasks:

$$TCR = \frac{N_{success}}{N_{total}} \quad (5)$$

where N_{total} is the total number of tasks and $N_{success}$ is the count of tasks where the final state meets the acceptance criteria. While navigation metrics (TUI/HPP) measure intermediate process correctness, TCR serves as the decisive metric for the agent’s practical utility in real-world scenarios.

Implementation Details The experiments are conducted on a server equipped with NVIDIA RTX 4090 GPUs. Multi-GPU parallelism is employed to accelerate inference. During the experiments, the web navigation process for the official documentation is executed for 15 steps, which is used to acquire task-relevant prior knowledge and construct candidate action strategies. Then, the task execution phase is carried out for 50 steps to validate and evaluate the effectiveness of the final strategy. All experiments are repeated under same hardware configurations to ensure fairness and reproducibility. The prompt used in the experiments is shown in Appendix F.

4.2. Main Results

The main experimental results of different GUI agents on the DocOS are shown in Table 3. The evaluation is conducted across three difficulty levels (Easy, Medium, and Hard).

All evaluated agents exhibit limited end-to-end task completion performance on DocOS. Most agents achieve limited TCR even on the Easy subset, and performance consistently degrades as task difficulty increases. These results indicate that DocOS presents a challenging evaluation setting that effectively exposes the limitations of existing

agents on long-tailed, application-specific, and multi-step GUI workflows. Meanwhile, several models maintain relatively stable TUI and HPP across difficulty levels, indicating that they are able to issue queries and make partial progress toward relevant documentation or target paths. However, such navigation-level progress frequently fails to translate into successful task completion. The experiments results suggest that the reliability of subsequent document-guided execution remain limited under the DocOS setting.

While the agents are generally able to access official documentation websites, they struggle to precisely localize task-relevant information within these websites. Across most agents and levels, the stable HPP scores suggest that agents are often able to reach higher-level sections of the target paths. However, the consistently low TCR implies that these agents frequently fail to identify the specific pages, sections within the documentations that directly correspond to the required operation. As a result, limitations in navigating within the documentation websites and locating the exact information that provides actionable guidance emerge as a major factor associated with low task completion performance.

5. Analysis

In this section, we conduct a comprehensive analysis to answer the following research questions **RQ1**: *What is the importance of documents in GUI agent performance?* (§5.1) **RQ2**: *What are the bottlenecks in the GUI agents' capabilities?* (§5.2) **RQ3**: *How do document length and step size affect task completion?* (§5.3) **RQ4**: *How does the GUI agent behave in case studies?* (§5.4)

5.1. RQ1: Importance of Documents

Table 4. Performance comparison of various GUI agents with and without access to external documents. The last column indicates the relative performance change (Δ_{rel}).

Models	w/o Document	w/ Document	$\Delta_{rel}^{\%}$
GELab-Zero-4B-preview	4.25	4.36(+0.11)	$\uparrow 2.59\%$
UI-TARS-1.5-7B	16.70	17.31(+0.61)	$\uparrow 3.65\%$
MAI-UI-8B	10.18	10.96(+0.78)	$\uparrow 7.66\%$
Aguvis-7B	1.91	1.95(+0.04)	$\uparrow 2.09\%$
GUI-Owl-7B	5.27	5.58(+0.31)	$\uparrow 5.88\%$
Qwen3-VL-8B	4.01	4.23(+0.22)	$\uparrow 5.49\%$

To verify the effectiveness of the Document-Guided Action task, we investigate the impact of introducing document guidance on the performance of GUI agents in task execution. Specifically, we have the agents skip the *Autonomous Knowledge Retrieval* phase as described in Section 3.1 and directly rely on the model’s parameterized knowledge to perform the tasks.

The results of this comparative analysis are presented in Table 4. As observed, integrating external documents generally enhances the performance of most GUI agents, validating the necessity of the Document-Guided Action task. In detail, **MAI-UI-8B** demonstrates the most significant benefit from document guidance, achieving a relative improvement of $\uparrow 7.67\%$, followed by **GUI-Owl-7B** with a $\uparrow 5.88\%$, and **Qwen3-VL-8B** with a $\uparrow 5.49\%$ increase. Notably, even the strongest baseline, **UI-TARS-1.5-7B**, which possesses robust parameterized knowledge, sees a performance gain of $\uparrow 3.67\%$, indicating that external documentation effectively supplements internal knowledge gaps even for advanced models. However, the impact varies across model capacities. **Aguvis-7B** shows less performance increase, suggesting a potential bottleneck in its ability to utilize long-context information. The smaller model, **GELab-Zero-4B-preview**, experiences a performance increase of $\uparrow 2.59\%$. This situation likely stems from the model’s limited context window or reasoning capacity, where the additional document content introduces noise or distraction rather than guidance. Overall, for models with sufficient comprehension capabilities, the introduction of documents serves as a critical bridge for accurate task execution.

5.2. RQ2: Bottleneck in GUI Agents

Table 5. Performance comparison of different models using the full pipeline (w/ Document) versus using ground-truth instructions (Oracle Document).

Models	w/ Document	Oracle Document	$\Delta_{rel}^{\%}$
GELab-Zero-4B-preview	4.36	4.83(+0.47)	$\uparrow 10.78\%$
UI-TARS-1.5-7B	17.31	17.98(+0.67)	$\uparrow 3.87\%$
MAI-UI-8B	10.96	12.04(+1.08)	$\uparrow 9.85\%$
Aguvis-7B	1.95	2.1(+0.15)	$\uparrow 7.69\%$
GUI-Owl-7B	5.58	5.81(+0.23)	$\uparrow 4.12\%$
Qwen3-VL-8B	4.23	4.44(+0.21)	$\uparrow 4.96\%$

In this section, we delve into the bottlenecks of the agents in the Document-Guided Action task. By eliminating the need for document retrieval, we directly evaluate the agent’s ability to execute tasks when provided with reliable, pre-provided instructions (i.e., an oracle document).

Table 5 reports the results of providing agents with Oracle Documents (ground-truth instructions) compared to the standard setting. The results reveal a divergent trend that highlights distinct bottlenecks across different models. On one hand, **MAI-UI-8B** and **GELab-Zero-4B-preview** exhibit substantial performance gains, with relative improvements of $\uparrow 9.85\%$ and $\uparrow 10.78\%$, respectively. This drastic increase suggests that the primary bottleneck for these models in the standard setting is the *accuracy of information acquisition*. When the noise of retrieval is removed and correct instructions are provided, these models demonstrate strong grounding and execution capabilities, proving they

are capable "followers" but weaker "planners."

On the other hand, unexpectedly, strong baselines like **UI-TARS-1.5-7B** ($\uparrow 3.87\%$) and **Qwen3-VL-8B** ($\uparrow 4.96\%$) experience a less significant performance increase when provided with Oracle documents. This counter-intuitive result points to a *context processing bottleneck*. Despite having the correct information, these models likely struggle to filter relevant actions from the detailed Oracle descriptions, or the lengthy context acts as a distraction that interferes with their inherent reasoning capabilities. This indicates that for some models, simply feeding "correct" knowledge is insufficient if the model lacks the robustness to handle long-context instructions effectively.

5.3. RQ3: Impact of the Document Length and Step Size

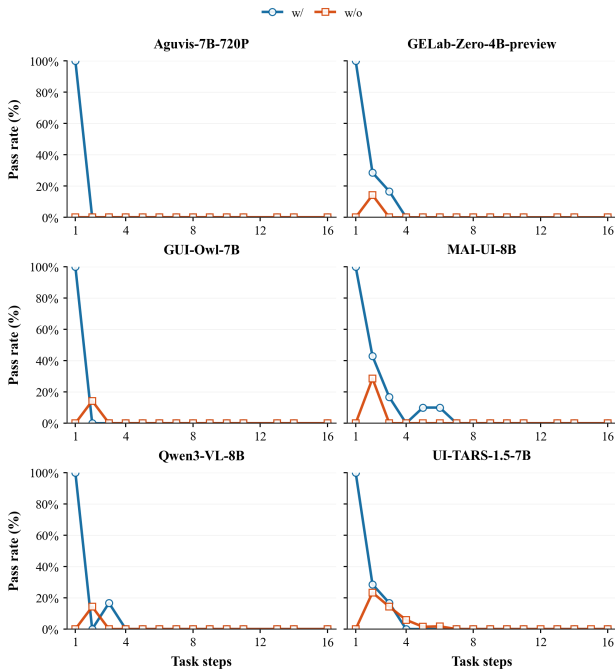


Figure 3. Pass rates of six different GUI agents across varying task steps under two settings: with (w/) and without (w/o) provided documents. The x-axis represents the number of steps required to complete the task, and the y-axis represents the pass rate.

In this section, we analyze how varying the length and step size of the documents provided to the GUI agent influences its task completion performance. As shown in Figure 3, we observe a significant negative correlation between the number of task steps and the agent’s success rate across all six evaluated models.

Specifically, in the setting with documents (w/), most models achieve a near 100% pass rate for single-step tasks. However, the performance degrades sharply as the task horizon extends. For tasks requiring more than 4 steps, the pass rate drops to nearly 0% for almost all models, indicating that

current agents struggle significantly with long-horizon planning and context retention, even when auxiliary documents are provided.

In contrast, the setting without documents (w/o) shows consistently lower performance. Notably, for 1-step tasks, the w/o setting often yields a near 0% pass rate, suggesting that without external guidance, agents fail to initiate the correct action for specific UI tasks.

5.4. RQ4: Error Analysis

To understand the limitation of current GUI agents in the document-guided action setting, We randomly sampled a subset of failed cases, conduct a detailed error analysis on them, and illustrate the primary error types during two stages. Detailed cases are in Appendix C.

During the document search and navigation stage, the errors arised can be categorized into several recurring patterns. **(1) Imprecise Localization** GUI agent fail to precisely localize task-relevant content within documentation. This is the most prominent and impactful failure mode in the failed cases. While agents are often able to reach the official documentation website and navigate to high-level pages, the frequently remain on loosely relevant section. **(2) Non-official Reference** In some failure cases, GUI agents retrieve information from third-party blogs, or tutorials instead of authoritative documentation. These sources often contain incomplete instructions, leading agents to perform incorrect actions. **(3) Execute before Retrieval** GUI agent bypasses documentation search and directly attempting task execution. Some failed cases ignore the guidance and proceed to execute the task immediately. Such behavior results in incorrect action sequences based on their parametric knowledge.

For GUI agents executing tasks based on retrieved search results, we find that the primary causes of failure are as follows. **(1) Action Grounding Failure** There is difficulty for GUI agent to translate instructions into concrete verifiable GUI actions. Even when agents access relevant interfaces, GUI agent stop at navigation without performing the specific action such as parameter adjustments. This reveals a gap between high-level instruction recognition and low-level action grounding. **(2) Context Misidentification** GUI Agent fail to identify the correct operational context. For example, it did not enter the required mode (e.g., Edit Mode) before attempting further actions. As a result, subsequent operations that depend on object mode were never triggered.

6. Conclusion

To improve the evaluation of GUI agents, we introduced Proactive Document-Guided Action, a document-driven paradigm that addressed the limitation of existing GUI agent evaluations by requiring agents to actively retrieve exter-

nal documentation when solving long-tailed, application-specific tasks. Within this paradigm, we proposed DocOS, a benchmark that evaluates GUI agents across the full pipeline of retrieval, understanding, and execution in realistic desktop environments. The experiment results show that GUI agents struggle to precisely localize task-relevant data and translate retrieved instructions into executable GUI actions, which provides valuable insights into the key bottlenecks that limit reliable, self-guided deployment of GUI agents. While these observations are contextualized within the specific architectures and evaluation benchmarks explored in our work, we provide a foundational understanding of the challenges that future GUI agents must overcome.

Impact Statement

DocOS aims to evaluate GUI agents under the novel paradigm of proactive document-guided action, which can enhance the reliability of GUI agents to resolve long-tailed tasks. We hope DocOS provides researchers with a systematic benchmark to better understand the strengths and limitations of GUI agents, and we strongly encourage responsible and ethical research and deployment of GUI agent systems.

Acknowledgments

Thanks for the insightful comments and feedback from the reviewers. This work was supported by the National Natural Science Foundation of China (No. 62406015). Jingjing Liu, Ziyi Huang, and Zihao Cheng contributed equally to this work. Zeming Liu is the corresponding author.

References

- Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., Ge, W., Guo, Z., Huang, Q., Huang, J., Huang, F., Hui, B., Jiang, S., Li, Z., Li, M., Li, M., Li, K., Lin, Z., Lin, J., Liu, X., Liu, J., Liu, C., Liu, Y., Liu, D., Liu, S., Lu, D., Luo, R., Lv, C., Men, R., Meng, L., Ren, X., Ren, X., Song, S., Sun, Y., Tang, J., Tu, J., Wan, J., Wang, P., Wang, P., Wang, Q., Wang, Y., Xie, T., Xu, Y., Xu, H., Xu, J., Yang, Z., Yang, M., Yang, J., Yang, A., Yu, B., Zhang, F., Zhang, H., Zhang, X., Zheng, B., Zhong, H., Zhou, J., Zhou, F., Zhou, J., Zhu, Y., and Zhu, K. Qwen3-vl technical report, 2025.
- Bu, W., Wu, Y., Yu, Q., Gao, M., Miao, B., Zhang, Z., Pan, K., Li, Y., Li, M., Ji, W., et al. What limits virtual agent application? omnibench: A scalable multi-dimensional benchmark for essential virtual agent capabilities. *arXiv preprint arXiv:2506.08933*, 2025.
- Cao, R., Lei, F., Wu, H., Chen, J., Fu, Y., Gao, H., Xiong, X., Zhang, H., Mao, Y., Hu, W., Xie, T., Xu, H., Zhang, D., Wang, S., Sun, R., Yin, P., Xiong, C., Ni, A., Liu, Q., Zhong, V., Chen, L., Yu, K., and Yu, T. Spider2-v: How far are multimodal agents from automating data science and engineering workflows?, 2024.
- Chen, A., Lou, L., Chen, K., Bai, X., Xiang, Y., Yang, M., Zhao, T., and Zhang, M. Benchmarking LLMs for translating classical Chinese poetry: Evaluating adequacy, fluency, and elegance. In Christodoulopoulos, C., Chakraborty, T., Rose, C., and Peng, V. (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 33019–33036, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.1678.
- Chen, D., Huang, Y., Wu, S., Tang, J., Chen, L., Bai, Y., He, Z., Wang, C., Zhou, H., Li, Y., et al. Gui-world: A video benchmark and dataset for multimodal gui-oriented understanding. *arXiv preprint arXiv:2406.10819*, 2024.
- Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., and Su, Y. Mind2web: Towards a generalist agent for the web, 2023.
- Fu, T., Su, A., Zhao, C., Wang, H., Wu, M., Yu, Z., Hu, F., Shi, M., Dong, W., Wang, J., Chen, Y., Yu, R., Peng, S., Li, M., Huang, N., Wei, H., Yu, J., Xin, Y., Zhao, X., Gu, K., Jiang, P., Zhou, S., and Wang, S. Mano technical report, 2025.
- Gou, B., Huang, Z., Ning, Y., Gu, Y., Lin, M., Yu, B., Kopanov, A., Qi, W., Shu, Y., Wu, J., Song, C. H., Gutierrez, B. J., Li, Y., Liao, Z., Moussa, H. N., ZHANG, T., Xie, J., Xue, T., Chen, S., Zheng, B., Zhang, K., Cai, Z., Rozgic, V., Ziyadi, M., Sun, H., and Su, Y. Mind2web 2: Evaluating agentic search with agent-as-a-judge. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025.
- Hong, W., Wang, W., Lv, Q., Xu, J., Yu, W., Ji, J., Wang, Y., Wang, Z., Zhang, Y., Li, J., Xu, B., Dong, Y., Ding, M., and Tang, J. Cogagent: A visual language model for gui agents, 2024.
- Jurmu, M., Boring, S., and Riekk, J. Screenspot: Multidimensional resource discovery for distributed applications in smart spaces. In *Proceedings of the 5th Annual International Conference on Mobile and Ubiquitous Systems: Computing, Networking, and Services*, pp. 1–9, 2008.
- Kapoor, R., Butala, Y. P., Russak, M., Koh, J. Y., Kamble, K., AlShikh, W., and Salakhutdinov, R. Omniact: A dataset and benchmark for enabling multimodal generalist autonomous agents for desktop and web. In *European*

- Conference on Computer Vision*, pp. 161–178. Springer, 2024.
- Koh, J. Y., Lo, R., Jang, L., Duvvur, V., Lim, M. C., Huang, P.-Y., Neubig, G., Zhou, S., Salakhutdinov, R., and Fried, D. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. *arXiv preprint arXiv:2401.13649*, 2024.
- Li, K., Ziyang, M., Lin, H., Luo, Z., Tian, Y., Ma, J., Huang, Z., and Chua, T.-S. Screenspot-pro: GUI grounding for professional high-resolution computer use. In *Workshop on Reasoning and Planning for Large Language Models*, 2025.
- Li, X., Kehai, C., Zhao, T., and Yang, M. End-to-end speech translation with adversarial training. In Wu, H., Cherry, C., Huang, L., He, Z., Liberman, M., Cross, J., and Liu, Y. (eds.), *Proceedings of the First Workshop on Automatic Simultaneous Translation*, pp. 10–14, Seattle, Washington, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.autosimtrans-1.2.
- Li, Z., Bai, X., LONG, Y., Chen, K., Zhang, Y., Wei, X., Li, J., and Zhang, M. XIFBench: Evaluating large language models on multilingual instruction following. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026.
- Liang, X., Xie, W., Li, J., Wang, W., Chen, Y., Chen, K., and Zhang, M. Tool learning via inference-time scaling and cycle verifier. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 24658–24671, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.1266.
- Lin, K. Q., Li, L., Gao, D., Wu, Q., Yan, M., Yang, Z., Wang, L., and Shou, M. Z. Videogui: A benchmark for gui automation from instructional videos. In *NeurIPS*, 2024.
- Liu, X., Qin, B., Liang, D., Dong, G., Lai, H., Zhang, H., Zhao, H., Iong, I. L., Sun, J., Wang, J., et al. Autoglm: Autonomous foundation agents for guis. *arXiv preprint arXiv:2411.00820*, 2024.
- Liu, Z., Wang, H., Niu, Z.-Y., Wu, H., Che, W., and Liu, T. Towards conversational recommendation over multi-type dialogs. In Jurafsky, D., Chai, J., Schluter, N., and Tetreault, J. (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 1036–1049, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.98.
- Liu, Z., Wang, H., Niu, Z.-Y., Wu, H., and Che, W. DuRec-Dial 2.0: A bilingual parallel corpus for conversational recommendation. In Moens, M.-F., Huang, X., Specia, L., and Yih, S. W.-t. (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 4335–4347, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.356.
- Liu, Z., Xu, J., Lei, Z., Wang, H., Niu, Z.-Y., and Wu, H. Where to go for the holidays: Towards mixed-type dialogs for clarification of user goals. In Muresan, S., Nakov, P., and Villavicencio, A. (eds.), *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1024–1034, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.73.
- Liu, Z., Bai, X., Chen, K., Chen, X., Li, X., Xiang, Y., Liu, J., Li, H.-D., Wang, Y., Nie, L., and Zhang, M. A survey on the feedback mechanism of llm-based ai agents. In Kwok, J. (ed.), *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25*, pp. 10582–10592. International Joint Conferences on Artificial Intelligence Organization, 8 2025. doi: 10.24963/ijcai.2025/1175. Survey Track.
- Lu, Y., Yu, Q., Wang, H., Liu, Z., Su, W., Liu, Y., Guo, Y., Liang, M., Wang, Y., and Wang, H. TransBench: Breaking barriers for transferable graphical user interface agents in dynamic digital environments. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 12464–12478, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.645.
- Nguyen, D., Chen, J., Wang, Y., Wu, G., Park, N., Hu, Z., Lyu, H., Wu, J., Aponte, R., Xia, Y., et al. Gui agents: A survey. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 22522–22538, 2025.
- Pan, Y., Kong, D., Zhou, S., Cui, C., Leng, Y., Jiang, B., Liu, H., Shang, Y., Zhou, S., Wu, T., et al. Webcanvas: Benchmarking web agents in online environments. *arXiv preprint arXiv:2406.12373*, 2024.
- Qin, Y., Ye, Y., Fang, J., Wang, H., Liang, S., Tian, S., Zhang, J., Li, J., Li, Y., Huang, S., et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.
- Rawles, C., Li, A., Rodriguez, D., Riva, O., and Lillicrap, T. Android in the wild: A large-scale dataset for android device control, 2023.

- Shi, X., Liu, Z., Wang, C., Leng, H., Xue, K., Zhang, X., and Zhang, S. MidMed: Towards mixed-type dialogues for medical consultation. In Rogers, A., Boyd-Graber, J., and Okazaki, N. (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8145–8157, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.453.
- Wang, L., Deng, Y., Zha, Y., Mao, G., Wang, Q., Min, T., Chen, W., and Chen, S. Mobileagentbench: An efficient and user-friendly benchmark for mobile LLM agents. *CoRR*, abs/2406.08184, 2024a. doi: 10.48550/ARXIV.2406.08184.
- Wang, S., Liu, W., Chen, J., Zhou, Y., Gan, W., Zeng, X., Che, Y., Yu, S., Hao, X., Shao, K., et al. Gui agents with foundation models: A comprehensive survey. *arXiv preprint arXiv:2411.04890*, 2024b.
- Wang, X., Wang, B., Lu, D., Yang, J., Xie, T., Wang, J., Deng, J., Guo, X., Xu, Y., Wu, C. H., Shen, Z., Li, Z., Li, R., Li, X., Chen, J., Zheng, B., Li, P., Lei, F., Cao, R., Fu, Y., Shin, D., Shin, M., Hu, J., Wang, Y., Chen, J., Ye, Y., Zhang, D., Du, D., Hu, H., Chen, H., Zhou, Z., Yao, H., Chen, Z., Gu, Q., Wang, Y., Wang, H., Yang, D., Zhong, V., Sung, F., Charles, Y., Yang, Z., and Yu, T. Opencua: Open foundations for computer-use agents, 2025.
- Wu, Z., Han, C., Ding, Z., Weng, Z., Liu, Z., Yao, S., Yu, T., and Kong, L. Os-copilot: Towards generalist computer agents with self-improvement, 2024a.
- Wu, Z., Wu, Z., Xu, F., Wang, Y., Sun, Q., Jia, C., Cheng, K., Ding, Z., Chen, L., Liang, P. P., and Qiao, Y. Os-atlas: A foundation action model for generalist gui agents, 2024b.
- Xie, B., Shao, R., Chen, G., Zhou, K., Li, Y., Liu, J., Zhang, M., and Nie, L. GUI-explorer: Autonomous exploration and mining of transition-aware knowledge for GUI agent. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5650–5667, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.282.
- Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T. J., Cheng, Z., Shin, D., Lei, F., et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.
- Xu, R., Ma, K., Yu, W., Zhang, H., Ho, J. C., Yang, C., and Yu, D. Retrieval-augmented gui agents with generative guidelines. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 17877–17886, 2025a.
- Xu, T., Chen, L., Wu, D.-J., Chen, Y., Zhang, Z., Yao, X., Xie, Z., Chen, Y., Liu, S., Qian, B., et al. Crab: Cross-environment agent benchmark for multimodal language model agents. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 21607–21647, 2025b.
- Xu, Y., Wang, Z., Wang, J., Lu, D., Xie, T., Saha, A., Sahoo, D., Yu, T., and Xiong, C. Aguis: Unified pure vision agents for autonomous gui interaction. In *Forty-second International Conference on Machine Learning*.
- Yan, H., Wang, J., Huang, X., Shen, Y., Meng, Z., Fan, Z., Tan, K., Gao, J., Shi, L., Yang, M., et al. Step-gui technical report. *arXiv preprint arXiv:2512.15431*, 2025.
- Yao, S., Chen, H., Yang, J., and Narasimhan, K. Web-shop: Towards scalable real-world web interaction with grounded language agents. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.
- Ye, J., Zhang, X., Xu, H., Liu, H., Wang, J., Zhu, Z., Zheng, Z., Gao, F., Cao, J., Lu, Z., et al. Mobile-agent-v3: Fundamental agents for gui automation. *arXiv preprint arXiv:2508.15144*, 2025.
- Yin, S., Fu, C., Zhao, S., Li, K., Sun, X., Xu, T., and Chen, E. A survey on multimodal large language models. *National Science Review*, 11(12):nwae403, 2024.
- Zhang, B., Shang, Z., Gao, Z., Zhang, W., Xie, R., Ma, X., Yuan, T., Wu, X., Zhu, S.-C., and Li, Q. Tongui: Internet-scale trajectories from multimodal web tutorials for generalized gui agents, 2025.
- Zhang, C., Yang, Z., Liu, J., Han, Y., Chen, X., Huang, Z., Fu, B., and Yu, G. Appagent: Multimodal agents as smartphone users, 2023.
- Zhou, H., Zhang, X., Tong, P., Zhang, J., Chen, L., Kong, Q., Cai, C., Liu, C., Wang, Y., Zhou, J., et al. Mai-ui technical report: Real-world centric foundation gui agents. *arXiv preprint arXiv:2512.22047*, 2025.
- Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., et al. Webarena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations*, 2024.

A. Details of DocOS Environment

A.1. Infrastructure

Inspired by OSWorld (Xie et al., 2024), we construct an interactive desktop environment to support the evaluation of GUI agents. The environment is managed through Docker for containerized deployment and leverages QEMU for system-level virtualization, enabling a fully controllable and flexible execution setting. Specifically, to reduce storage overhead, improve execution efficiency, and enable flexible composition of software environments, we construct lightweight and modular filesystem images, each encapsulating a single target application along with its required dependencies.

A.2. Web Search

DocOS environment provides controlled Internet access to support instruction-driven online information retrieval. Network connectivity is enabled, and all web browsing and search activities are conducted exclusively through the Microsoft Edge browser, ensuring a realistic and consistent user interaction paradigm. During the search process, agents are required to formulate queries in a human-like manner, using natural-language keywords rather than structured search APIs or backend retrieval interfaces. In addition, the environment supports persistent recording of browsing states. Opened web pages are saved and screenshots are captured during task execution, providing visual records for result verification, behavior analysis, and reproducibility of the evaluation process.

A.3. Observation Space

The observation space is defined by screen captures in DocOS, enabling the evaluation of GUI agents based on visual perception. Screen captures provide a complete visual snapshot of the desktop environment, including window layouts, widget appearances, textual content, and graphical elements, supplying all necessary visual cues for interface understanding and action decision-making.

A.4. Action Space

Table 6. Detailed Specification of the DocOS Action Space. This table enumerates all atomic actions available to the agent. For each action type, we specify the required and optional (*opt*) parameters, their data types, valid ranges (based on a 1920×1080 screen resolution), and a functional description.

Action Type	Parameters	Description
MOVE_TO	x: float $\in [0, 1920]$ y: float $\in [0, 1080]$	Move the cursor to the specified position.
CLICK	button: str $\in \{\text{left, right, middle}\}$ (opt) x: float $\in [0, 1920]$ (opt) y: float $\in [0, 1080]$ (opt) num_clicks: int $\in \{1, 2, 3\}$ (opt)	Click the specified button (default: left). Click at current position if x, y are not specified.
MOUSE_DOWN	button: str $\in \{\text{left, right, middle}\}$ (opt)	Press the specified button (default: left) without releasing.
MOUSE_UP	button: str $\in \{\text{left, right, middle}\}$ (opt)	Release the specified button (default: left).
RIGHT_CLICK	x: float $\in [0, 1920]$ (opt) y: float $\in [0, 1080]$ (opt)	Right click at the current position if x and y are not specified, otherwise at the specified position.
DOUBLE_CLICK	x: float $\in [0, 1920]$ (opt) y: float $\in [0, 1080]$ (opt)	Double click at the current position if x and y are not specified, otherwise at the specified position.

Action Type	Parameters	Description
DRAG_TO	x: float $\in [0, 1920]$ y: float $\in [0, 1080]$	Drag the cursor to the specified position with the left button pressed.
SCROLL	dx: int dy: int	Scroll the mouse wheel up or down.
TYPING	text: str	Type the specified text string.
PRESS	key: str $\in \text{KEYS}$	Press the specified key and release it.
KEY_DOWN	key: str $\in \text{KEYS}$	Press the specified key (hold down).
KEY_UP	key: str $\in \text{KEYS}$	Release the specified key.
HOTKEY	keys: list[str] $\subseteq \text{KEYS}$	Press the specified key combination simultaneously.
WAIT	<i>None</i>	Wait until the next action.
FAIL	<i>None</i>	Decide the task cannot be performed.
DONE	<i>None</i>	Decide the task is done.

We define the action space of **DocOS** by drawing inspiration from the design paradigm of OSWorld (Xie et al., 2024). The space consists of **16 atomic actions** that encompass a comprehensive set of mouse and keyboard interactions, enabling the agent to operate the GUI like a human user. All low-level executions are implemented using the `pyautogui`² library to ensure robust and cross-platform compatibility. The detailed specifications of these actions, including their parameters and functional descriptions, are provided in Table 6.

B. Details of DocOS Benchmark

B.1. Data Statistic

As shown in table 7, we collected updating document from official websites of a diverse set of real-world software applications, spanning development environments, multimedia tools, collaboration platforms, and dataa analytics systems. For each application, we report the corresponding documentation domain, the average number of steps, and the average prompt length.

B.2. Data Example

We provide detailed data examples in DocOS, including Figures 4-17. Each item is constructed with a task instruction, an expected document url, and several standard action steps. These examples cover a variety of document-grounded tasks with different levels of complexity.

C. Error cases

This section presents qualitative cases of common error types made by GUI agents. Figures 18, 19, and 20 show the primary error types in the process of proactive document search. Figures 21 and 22 further illustrate the main error types encountered during the document-grounded execution.

²<https://github.com/asweigart/pyautogui>

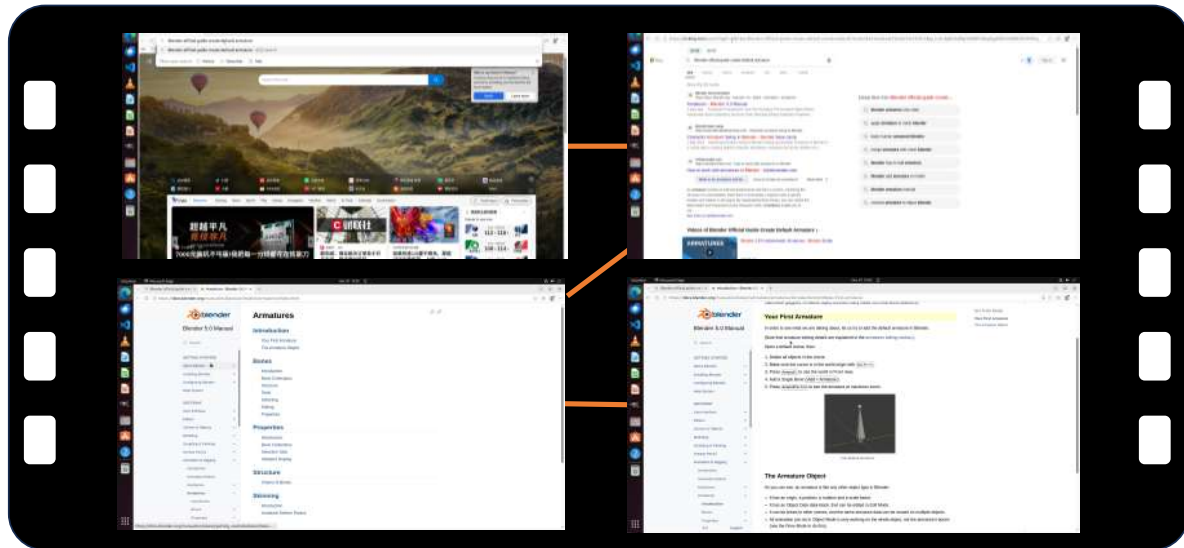
Table 7. Statistics of Tasks across Different Applications. The table lists the application name, its corresponding domain, total task count, average steps to completion, and average prompt length.

Application	Domain	Tasks	Avg. Steps	Avg. Prompt Len.
PyCharm	https://www.jetbrains.com/help/pycharm/	257	6.92	1479.28
Idea	https://www.jetbrains.com/help/idea	163	6.45	1260.59
Blender	https://docs.blender.org/manual/en/latest/	125	5.16	975.52
Postman	https://learning.postman.com/docs/	94	7.23	1292.50
Godot	https://docs.godotengine.org/	41	7.29	3020.51
Zotero	https://www.zotero.org/support/	39	6.03	1125.85
Zulip	https://zulip.com/help/	20	7.75	819.55
Element	https://docs.element.io/latest/	13	8.31	965.69
NetBeans	https://netbeans.apache.org/tutorial/	13	9.75	3862.08
VSCoDe	https://code.visualstudio.com/docs/	12	8.42	3616.75
Anki	https://docs.ankiweb.net/	8	4.62	1413.50
Odoo_19_0	https://www.odoo.com/documentation/19.0/	7	9.86	1502.29
Grafana	https://grafana.com/docs/grafana/	6	12.17	1172.33
Emby	https://emby.media/support/	4	6.50	1109.00
Metabase	https://www.metabase.com/docs/	4	7.00	1305.50
Cryptomator	https://docs.cryptomator.org/	3	3.33	897.67
Notepad++	https://npp-user-manual.org/docs/	3	8.00	4206.33
Signal	https://support.signal.org/	3	8.33	759.67
Geany	https://www.geany.org/	1	10.00	1321.00
VLC	https://wiki.videolan.org/	1	8.00	1399.00



Figure 4. A sample of Anki.

App: Blender



Task Instruction: Create a default armature in Blender by clearing the scene, centering the 3D cursor, switching to Front view, adding a single bone, and zooming in.

actual_url: <https://docs.blender.org/manual/en/latest/animation/armatures/introduction.html#your-first-armature>

expected_url: <https://docs.blender.org/manual/en/latest/animation/armatures/introduction.html>

Steps:

- Step_1: Delete all objects in the scene.
- Step_2: Ensure the 3D cursor is at the world origin using Shift-C.
- Step_3: Press Numpad1 to switch to Front view.
- Step_4: Add a Single Bone by choosing Add → Armature.
- Step_5: Press NumpadPeriod to maximize the zoom so the armature is fully visible.

Figure 5. A sample of Blender.

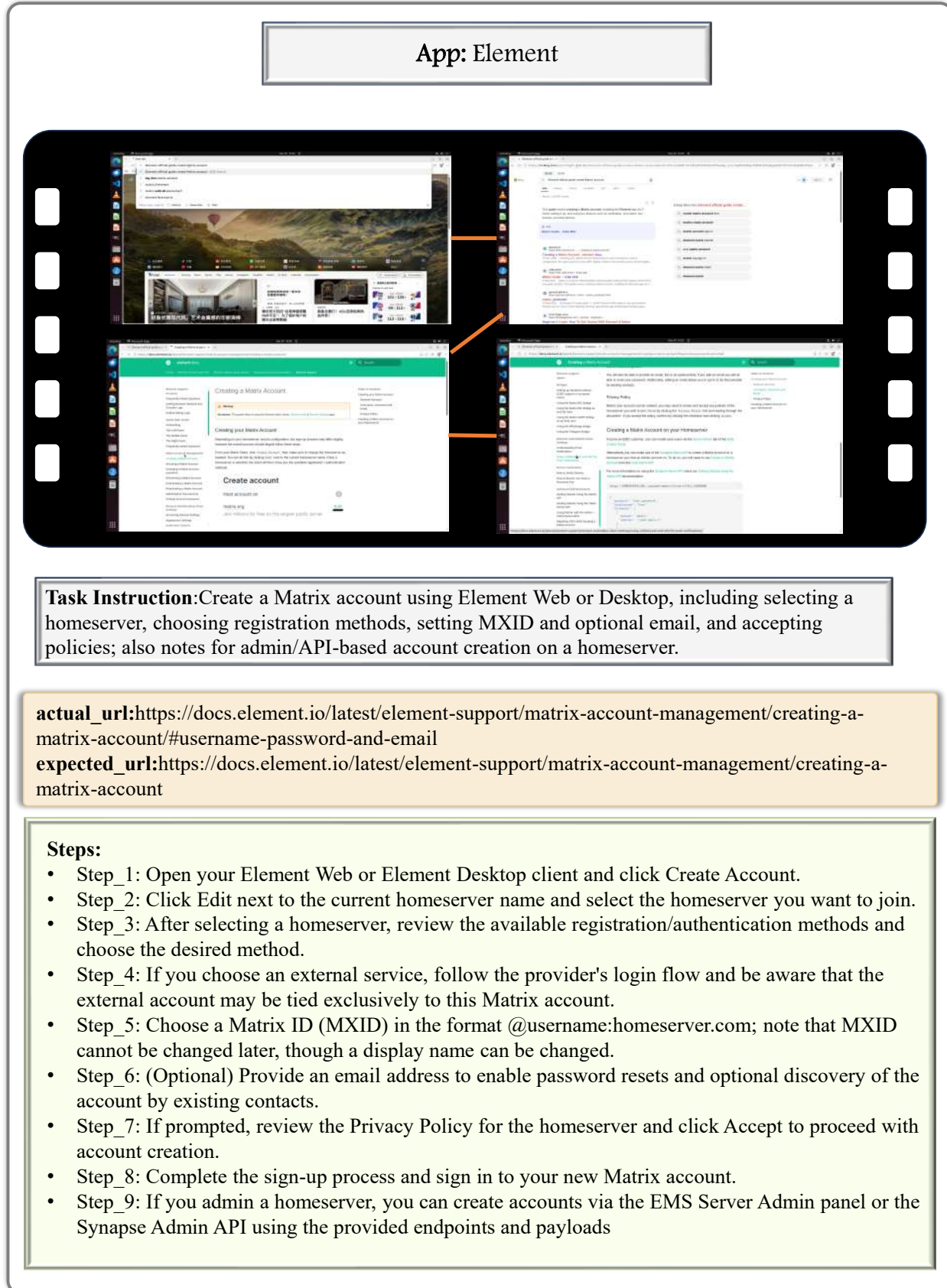
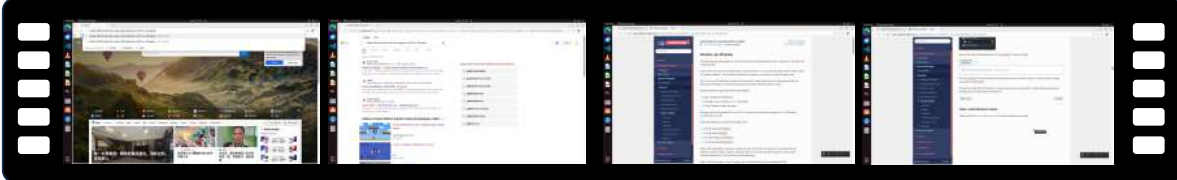


Figure 6. A sample of Element.

App: Godot



Task Instruction: Create and integrate a Heads-Up Display (HUD) in a Godot 2D game, with ScoreLabel, Message, StartButton, and MessageTimer, including font setup, layout anchors, signals, and game-state interactions.

actual_url:https://docs.godotengine.org/en/stable/getting_started/first_2d_game/06.heads_up_display.html

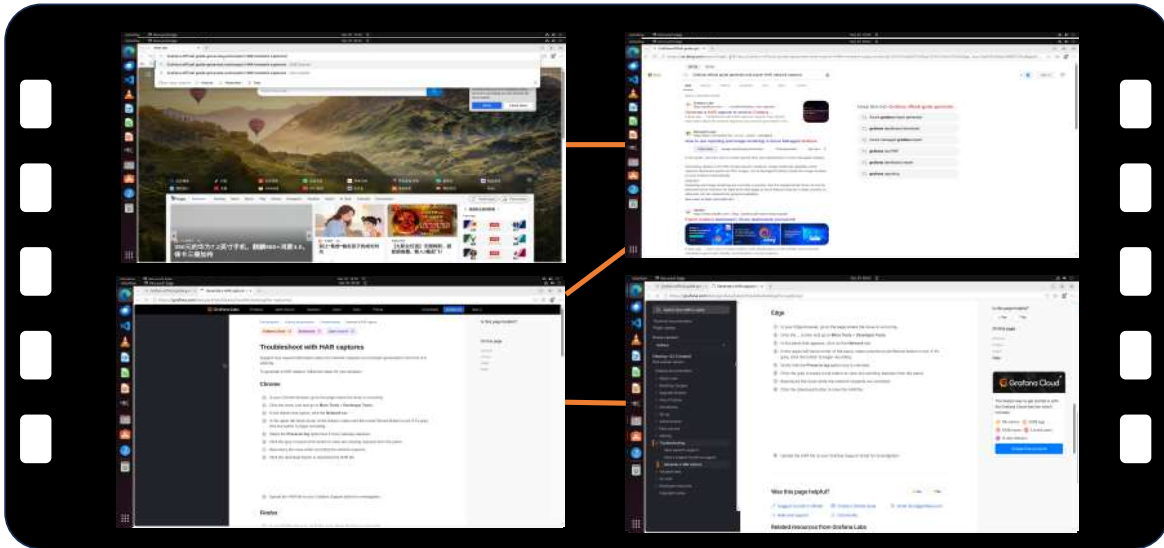
expected_url:https://docs.godotengine.org/en/stable/getting_started/first_2d_game/06.heads_up_display.html

Steps:

- Step_1: Create a new scene, add a CanvasLayer node named HUD to serve as the UI overlay, and add child nodes ScoreLabel, Message, StartButton, and MessageTimer.
- Step_2: Under HUD, ensure the children are ScoreLabel (Label), Message (Label), StartButton (Button), and MessageTimer (Timer).
- Step_3: Configure ScoreLabel: set initial text to 0, set Horizontal and Vertical Alignment to Center, load the Xolonium-Regular.ttf font via Theme Overrides > Fonts, and set font size to 64 via Theme Overrides > Font Sizes; use the Center Top anchor preset.
- Step_4: Configure Message: set text to 'Dodge the Creeps!', center alignments, enable Word autowrap, set Size X to 480 under Control - Layout/Transform, and use Center anchor preset.
- Step_5: Configure StartButton: set text to 'Start', set Size X to 200 and Size Y to 100, use Center Bottom anchor preset, and set Position Y to 580.
- Step_6: On MessageTimer, set Wait Time to 2 and One Shot to On.
- Step_7: Add a script to HUD implementing a start_game signal; declare the signal and ensure the project is rebuilt so the editor recognizes it.
- Step_8: Implement show_message(text) in HUD to set the Message text, show the Message, and start the MessageTimer.
- Step_9: Implement show_game_over() in HUD to display 'Game Over', wait for the MessageTimer timeout, then show 'Dodge the Creeps!' for 1 second, and finally reveal the StartButton.
- Step_10: Implement update_score(score) in HUD to update ScoreLabel with the given score.
- Step_11: Connect StartButton pressed() and MessageTimer timeout() signals to HUD; implement on_start_button_pressed() to hide StartButton and emit start_game, and on_message_timer_timeout() to hide the Message.
- Step_12: Instance the HUD scene in Main and connect HUD.start_game to Main.new_game(); verify a green connection appears next to the function in the script.
- Step_13: In new_game(), call HUD.update_score(score) and HUD.show_message('Get Ready').
- Step_14: In game_over(), call HUD.show_game_over().
- Step_15: In on_score_timer_timeout(), call HUD.update_score(score) to keep the HUD in sync with the score.
- Step_16: Remove any automatic start call from _ready() if present, then run the project to test and fix any issues.
- Step_17: To clean up between games, assign mobs to a 'mobs' group and, in Main.new_game(), call get_tree().call_group('mobs','queue_free') to remove existing creep.

Figure 7. A sample of Godot.

App: Grafana



Task Instruction: Generate and export HAR network captures from the browser's Developer Tools for troubleshooting Grafana issues.

actual_url: <https://grafana.com/docs/grafana/latest/troubleshooting/har-captures/>
expected_url: <https://grafana.com/docs/grafana/latest/troubleshooting/har-captures>

Steps:

- Step_1: In your Edge browser, go to the page where the issue is occurring.
- Step_2: Click the ... button and go to More Tools > Developer Tools.
- Step_3: In the panel that appears, click on the Network tab.
- Step_4: In the upper left hand corner of the panel, make sure the round Record button is red. If it's grey, click the button to begin recording
- Step_5: Verify that the Preserve log option box is checked.
- Step_6: Click the grey crossed circle button to clear any existing requests from the panel.
- Step_7: Reproduce the issue while the network requests are recorded.
- Step_8: Click the download button to save the HAR file.
- Step_9: Upload the HAR file to your Grafana Support ticket for investigation.

Figure 8. A sample of Grafana.

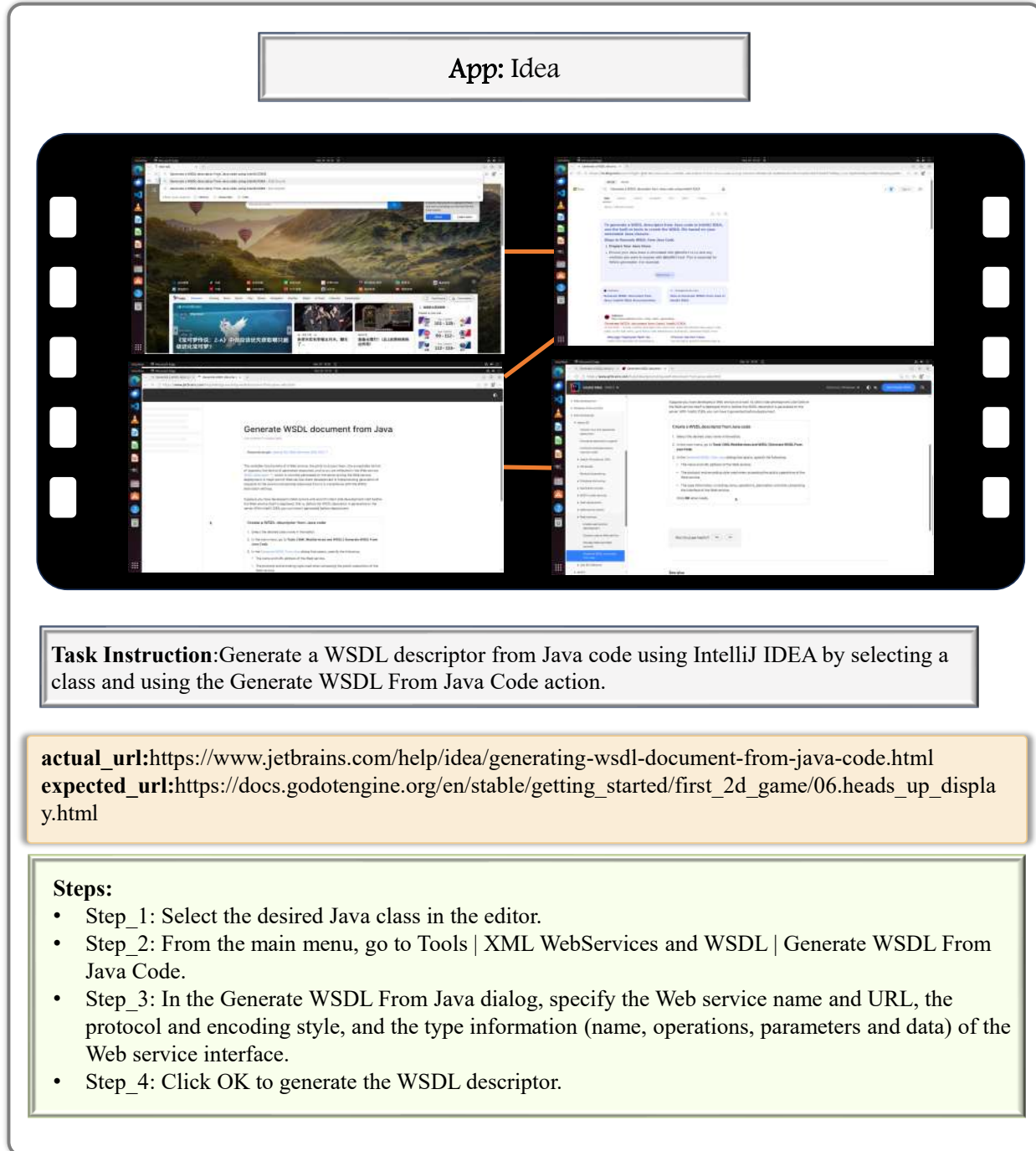


Figure 9. A sample of Idea.

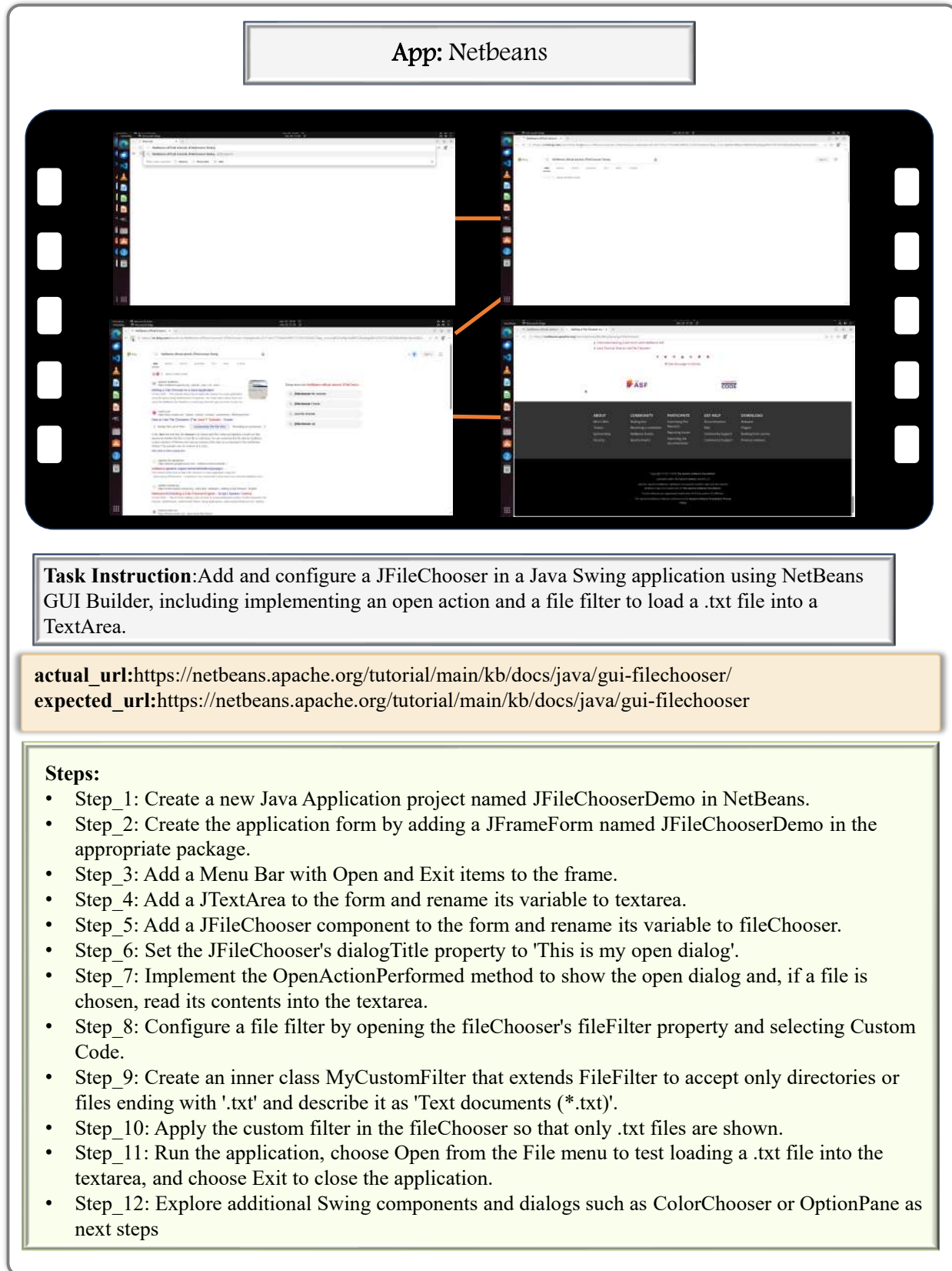
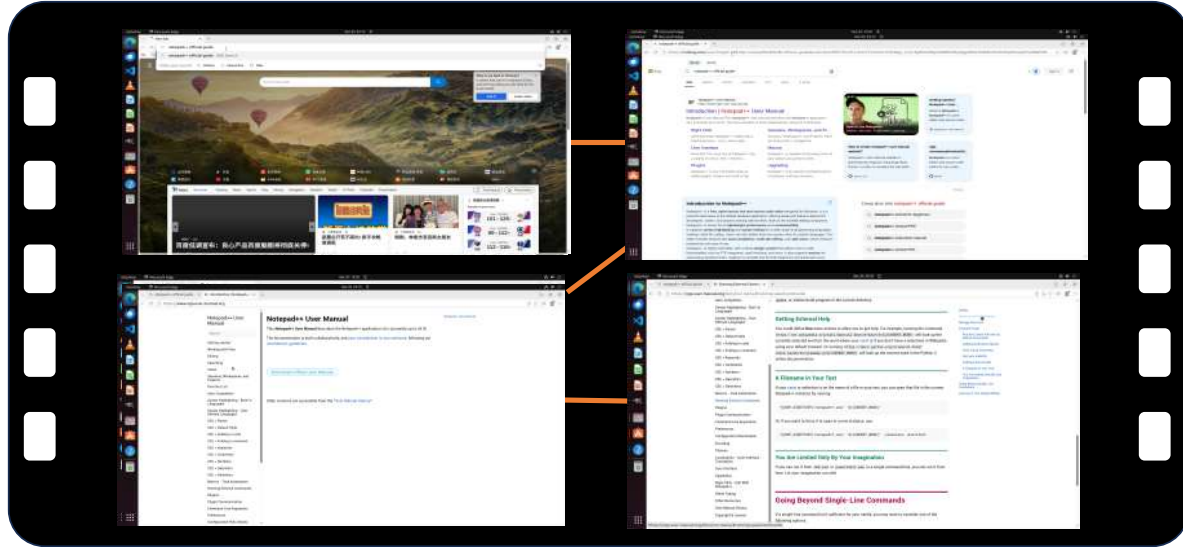


Figure 10. A sample of Netbeans.

App: Notepad++



Task Instruction: Use Notepad++ Run... to execute commands, save and manage Run entries, and run commands with examples.

actual_url: <https://npp-user-manual.org/docs/run-menu/#running-saved-commands>

expected_url: <https://npp-user-manual.org/docs/run-menu>

Steps:

- Step_1: Open Notepad++ and select Run > Run... to open the Run dialog.
- Step_2: In the Program to Run field, type a command or use the ... button to browse for an executable to run.
- Step_3: If needed, prefix the command with cmd.exe /c to run it in a Windows command prompt, or cmd.exe /k to keep the window open after execution; specify powershell.exe for PowerShell commands.
- Step_4: If the command is not in PATH, provide a full path (enclose paths with spaces in quotes, e.g. \"C:\\Program Files\\App\\app.exe\" \"arguments\").
- Step_5: Use the pulldown to reuse previous commands and click Run to execute; click Save... to save the command as a named Run entry and optionally assign a keyboard shortcut.
- Step_6: Click Cancel or the X to exit the dialog without running.
- Step_7: To run saved commands, select them under Run... in the menu; use Run > Modify Shortcut / Delete Command to manage shortcuts or remove commands.
- Step_8: See examples: run the current file with its default association using \"\$(FULL_CURRENT_PATH)\"; open a file in a browser or other app by saving a corresponding command (e.g., a browser executable with \"\$(FULL_CURRENT_PATH)\").
- Step_9: Be mindful of security: running external programs can affect your PC; only run commands from trusted sources

Figure 11. A sample of Notepad++.

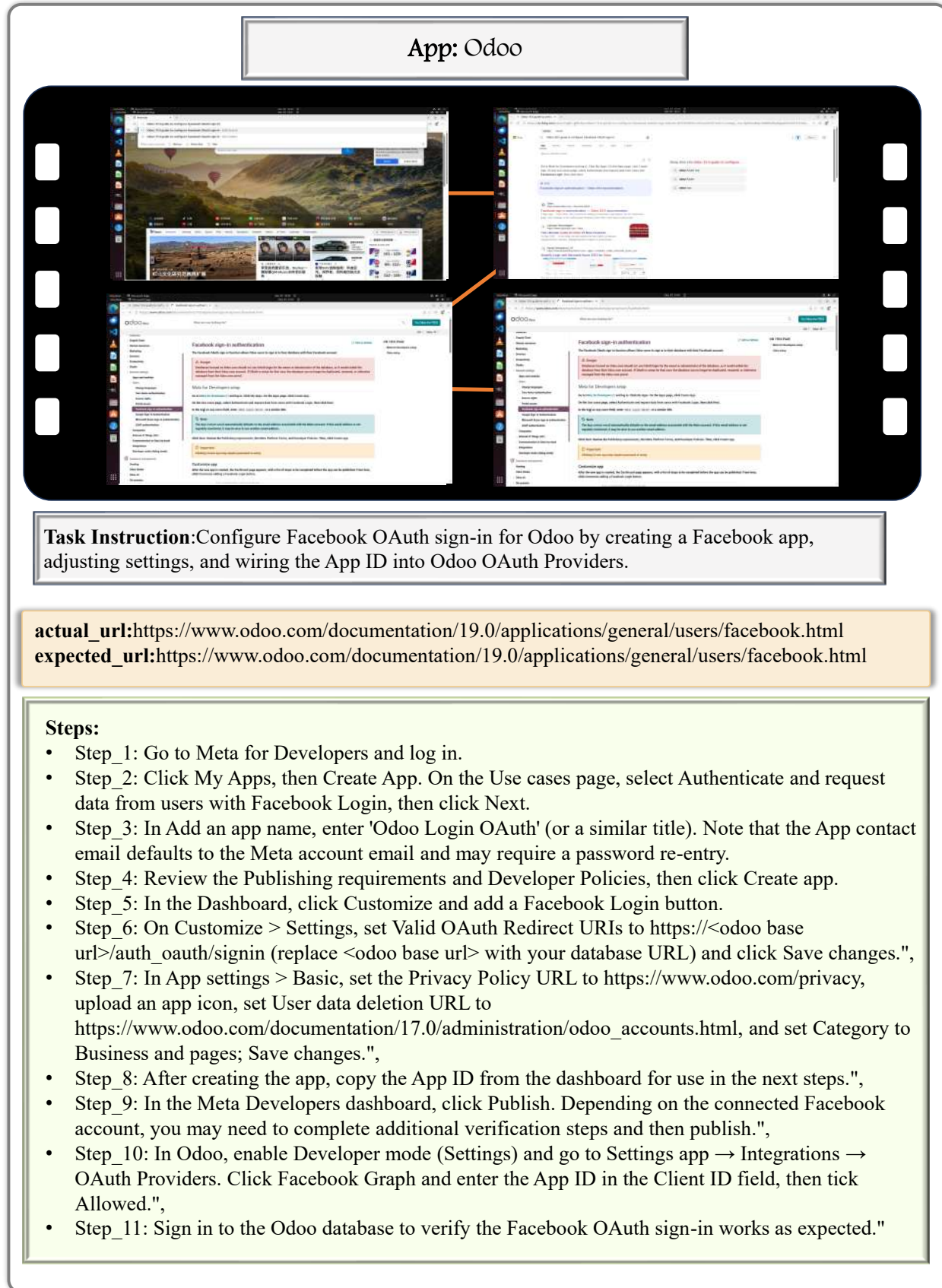


Figure 12. A sample of Odoo.

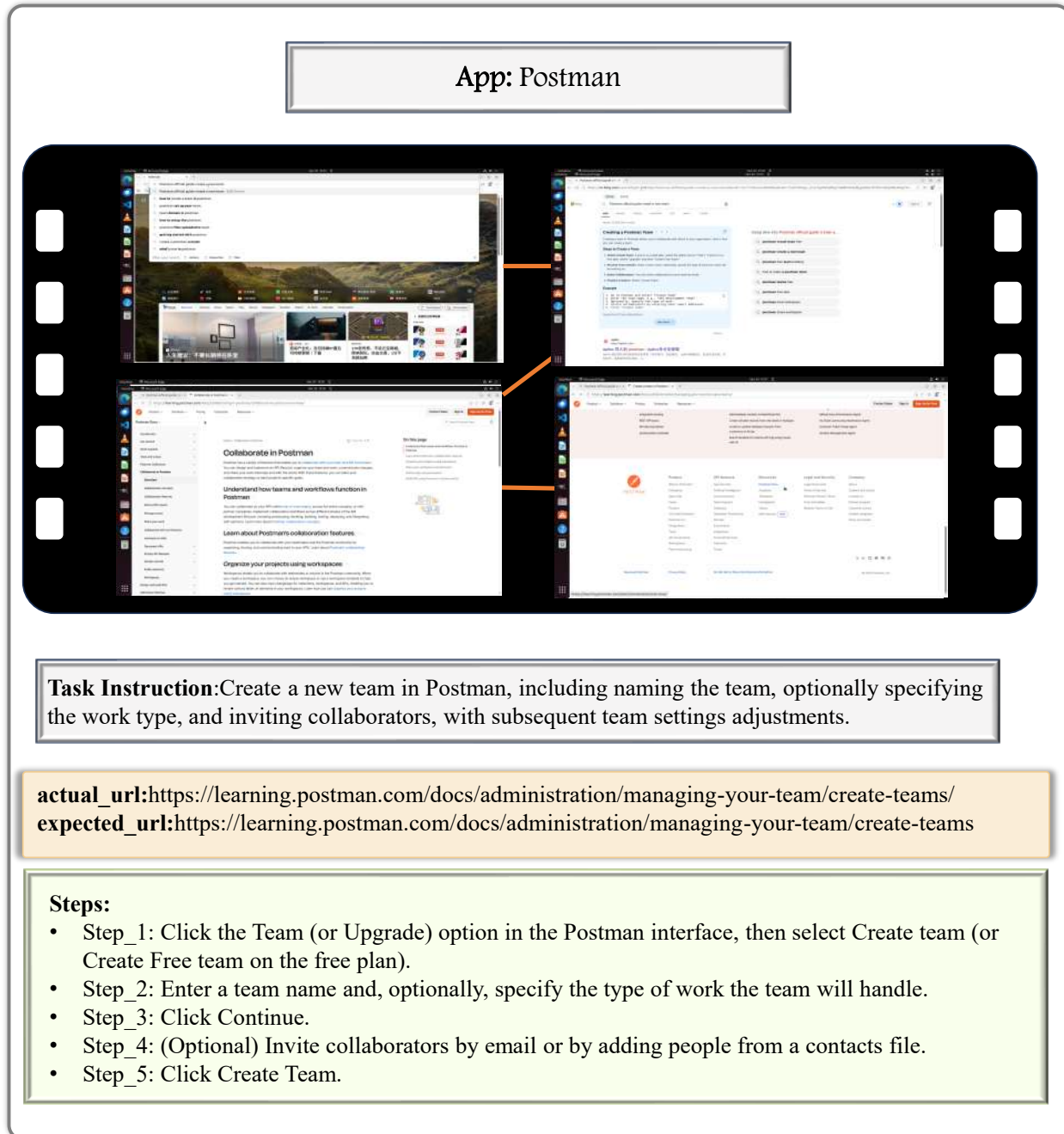


Figure 13. A sample of Postman.

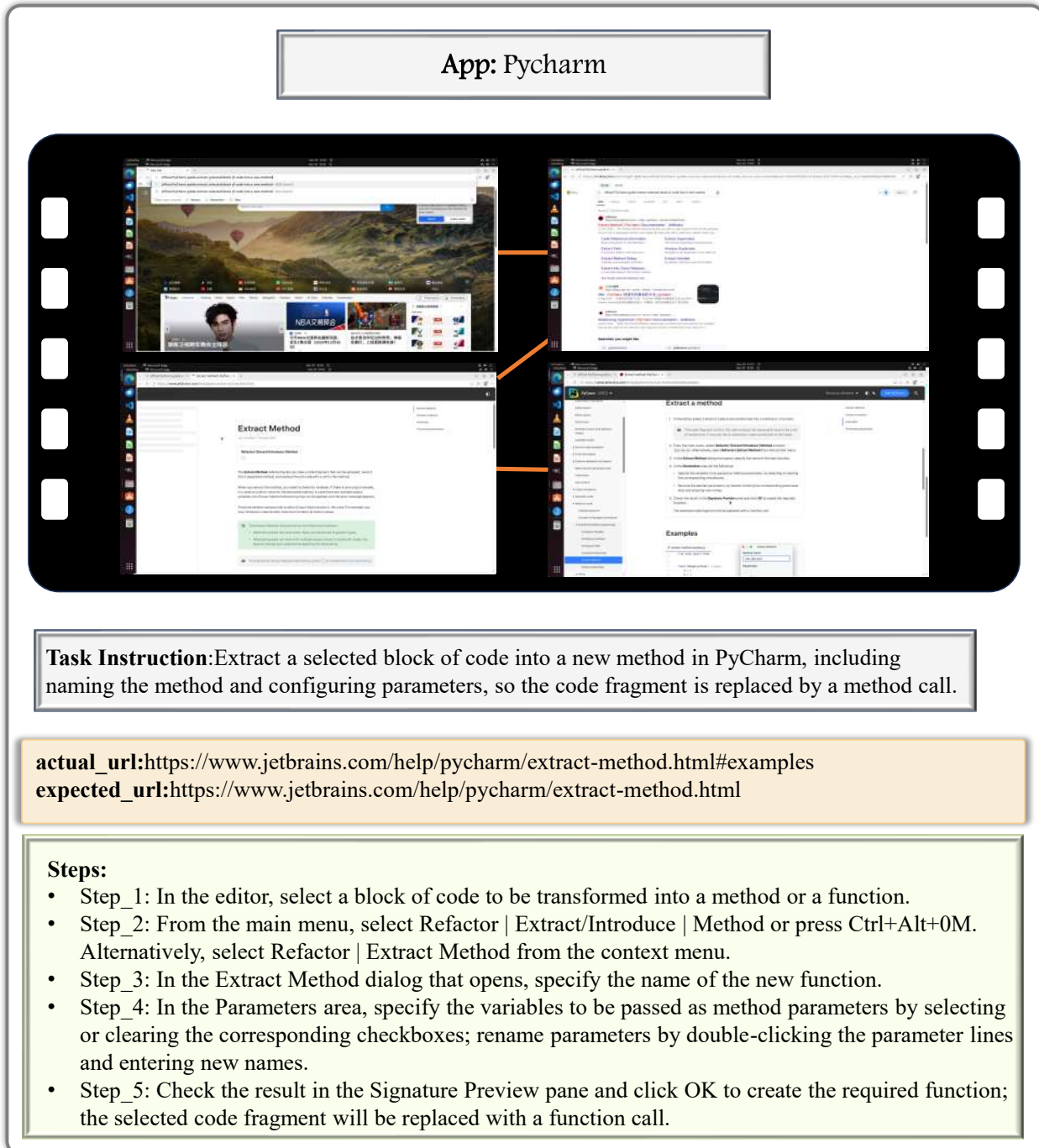


Figure 14. A sample of PyCharm.

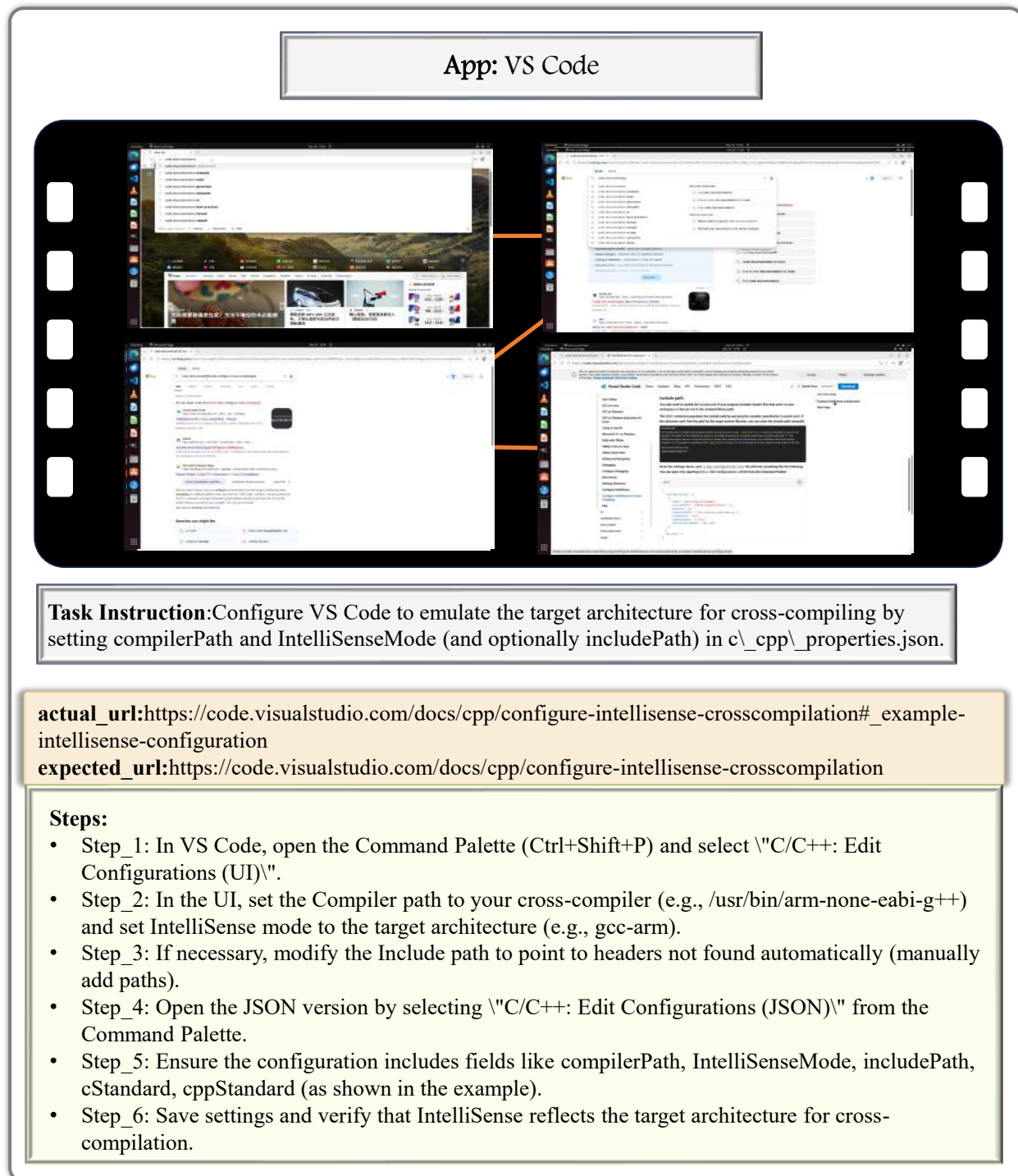
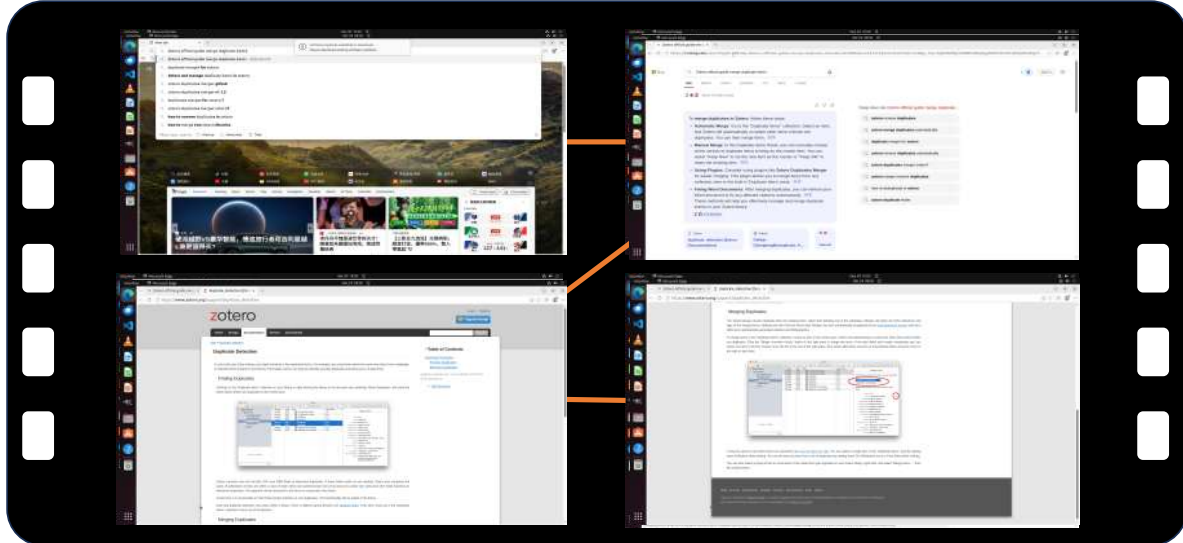


Figure 15. A sample of VSCode.

App: Zotero



Task Instruction: Locate and merge duplicate items in Zotero using the Duplicate Items view, including selecting duplicates, choosing a master item, and merging, with optional guidance for merging via context menu.

actual_url: https://www.zotero.org/support/duplicate_detection
expected_url: https://www.zotero.org/support/duplicate_detection

Steps:

- Step_1: Open Zotero and click the 'Duplicate Items' collection in the left pane, or right-click your library and select 'Show Duplicates' to display potential duplicates in the center pane.
- Step_2: Review the items listed as duplicates; Zotero uses title, DOI, and ISBN to identify duplicates, and may also consider year and overlapping authors when necessary.
- Step_3: In the Duplicate Items view, select one of the duplicate items in the center pane; Zotero will automatically co-select the other items it thinks are duplicates.
- Step_4: Click the 'Merge <number> Items' button in the right pane to merge the selected items into a single item.
- Step_5: If the item fields don't match completely, choose one item to be the 'master' from the list at the top of the right pane, then use the icons to the right of each field to select alternative versions for mismatched fields.
- Step_6: (Optional) Sort the items by Title to make it easier to see duplicates; you can select a single item in the 'Duplicate Items' view by holding down Alt/Option while clicking.
- Step_7: To deselect an item from a set of duplicates, hold down Ctrl (Windows/Linux) or Cmd (Mac) while clicking.
- Step_8: You can also select two or more items of the same item type anywhere in your Zotero library, right-click, and choose 'Merge Items...' from the context menu

Figure 16. A sample of Zotero.

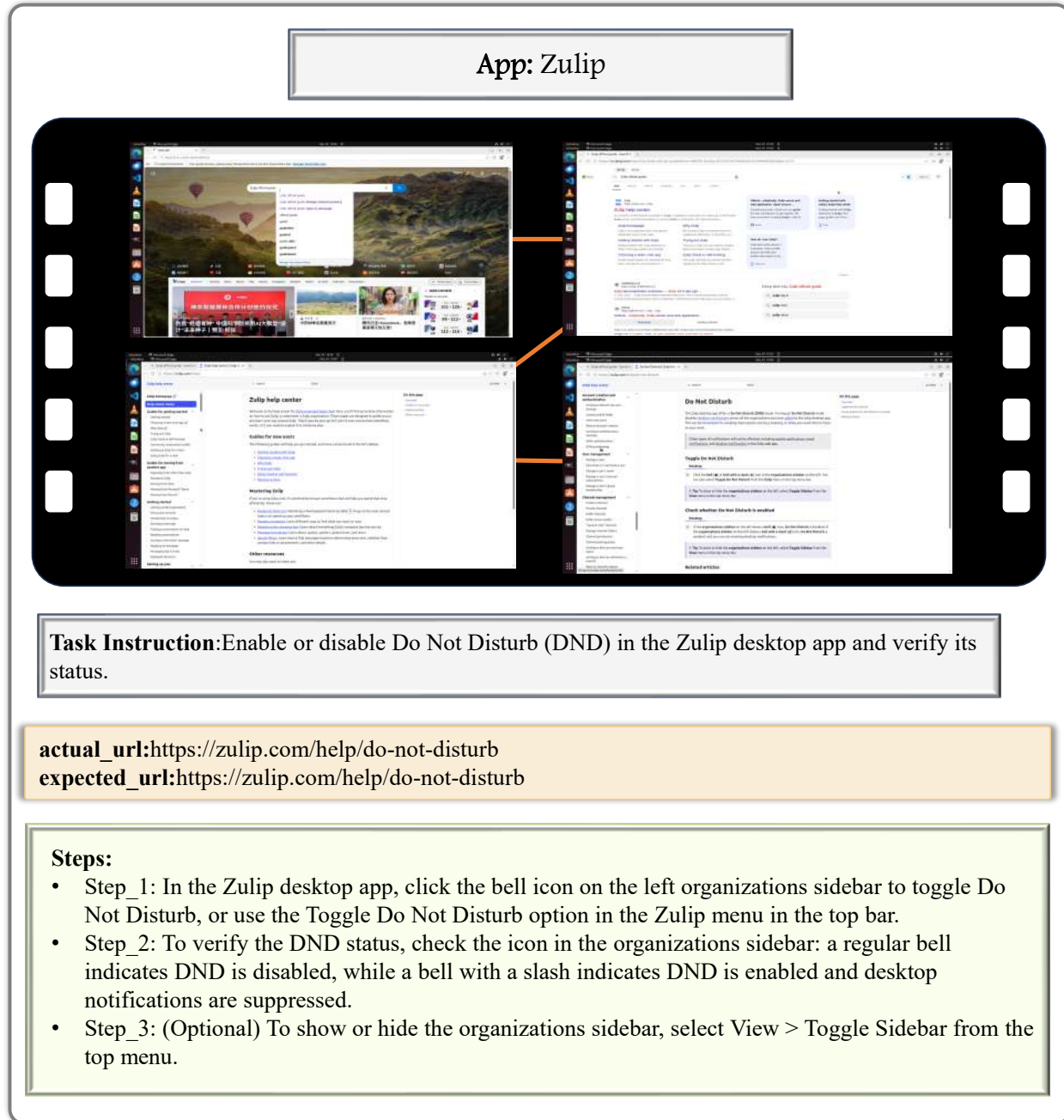
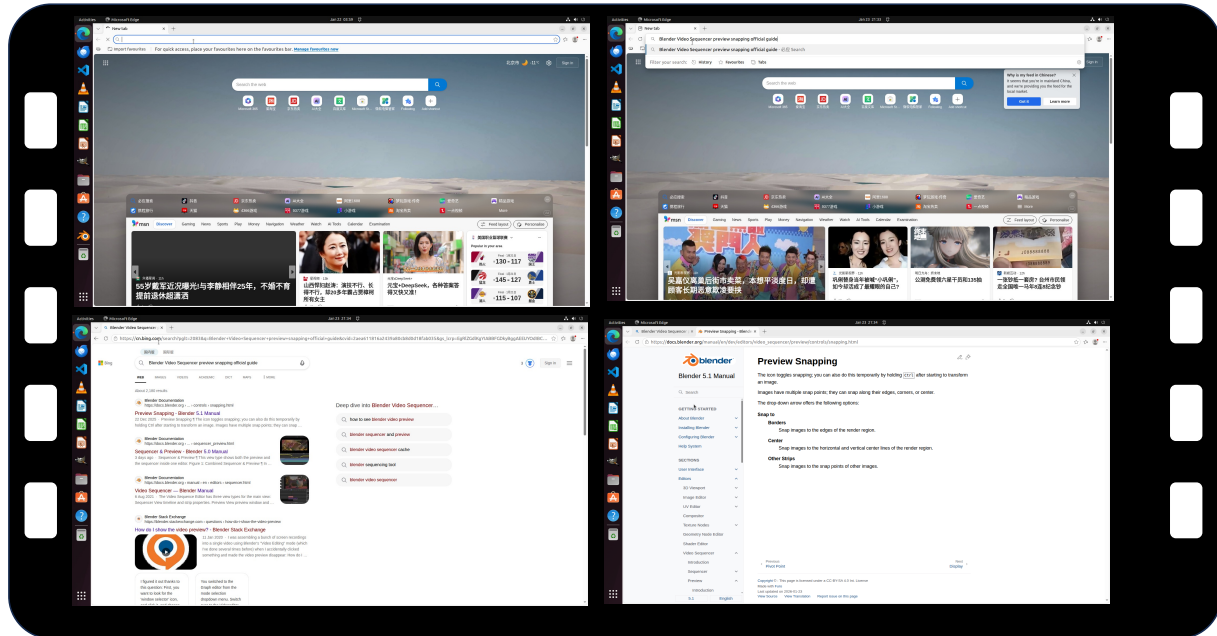


Figure 17. A sample of Zulip.

Error Type: Imprecise Localization

Task Instruction: Enable and use preview snapping in Blender's Video Sequencer to snap images to borders, center, or other strips, including enabling/disabling snapping and temporary Ctrl-based snapping.

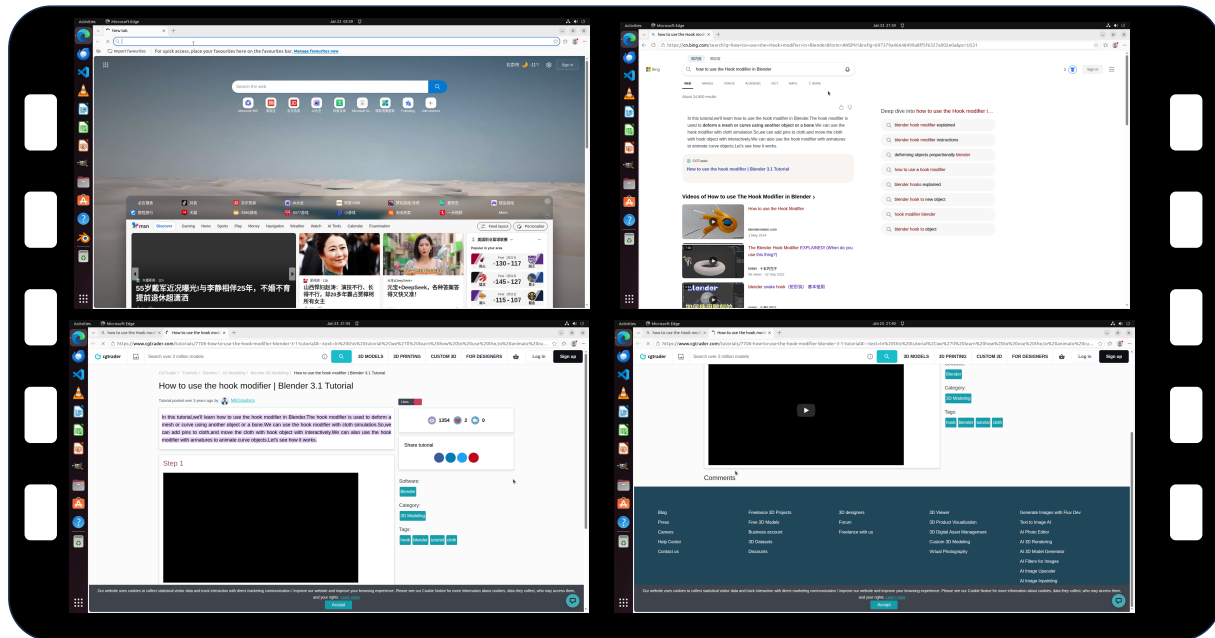


actual_url: https://docs.blender.org/manual/en/dev/editors/video_sequencer/preview/controls/snapping.html
expected_url: https://docs.blender.org/manual/en/latest/compositing/types/filter/sun_beams.html

Figure 18. An error case of imprecise localization.

Error Type: Non-official Reference

Task Instruction: Explain how to use the Hook modifier in Blender to deform a mesh by linking it to another object, including vertex assignment, falloff, and an example with an Empty.

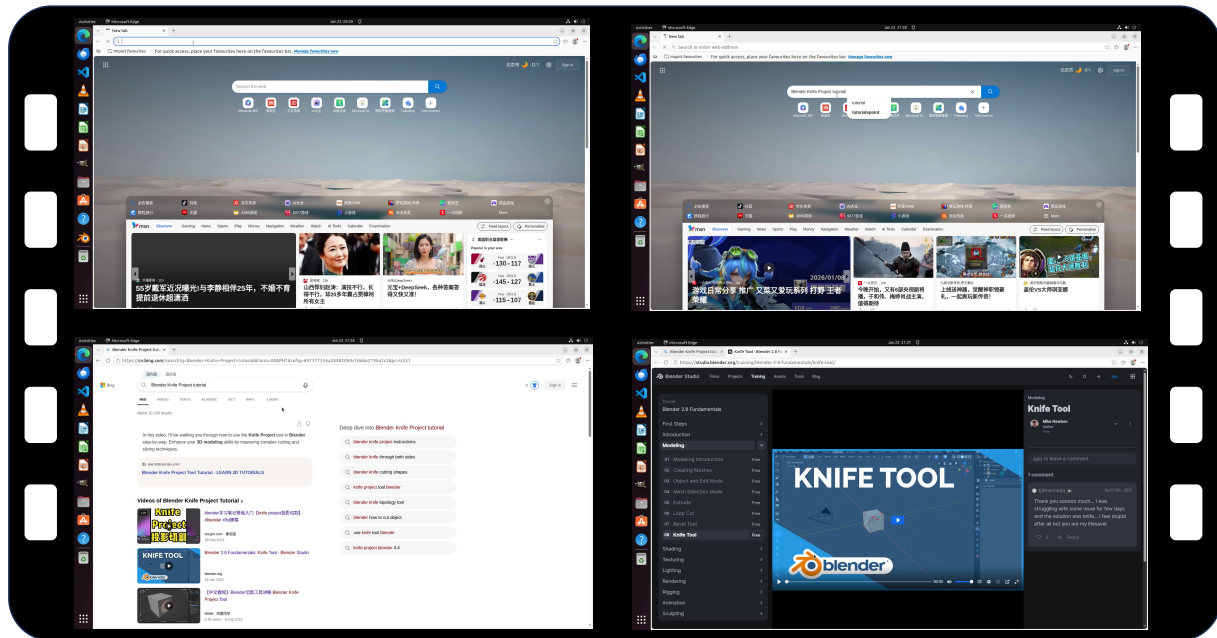


actual_url: <https://www.cgtrader.com/tutorials/7706-how-to-use-the-hook-modifier-blender-3-1-tutorial#:~:text=...>
expected_url: <https://docs.blender.org/manual/en/latest/modeling/modifiers/deform/hooks.html>

Figure 19. An error case of Non-official Reference.

Error Type: Execute Before Retrieval

Task Instruction: Guide to use Blender's Knife Project tool to project the outline of non-editing objects onto meshes in Edit Mode and cut them accordingly.

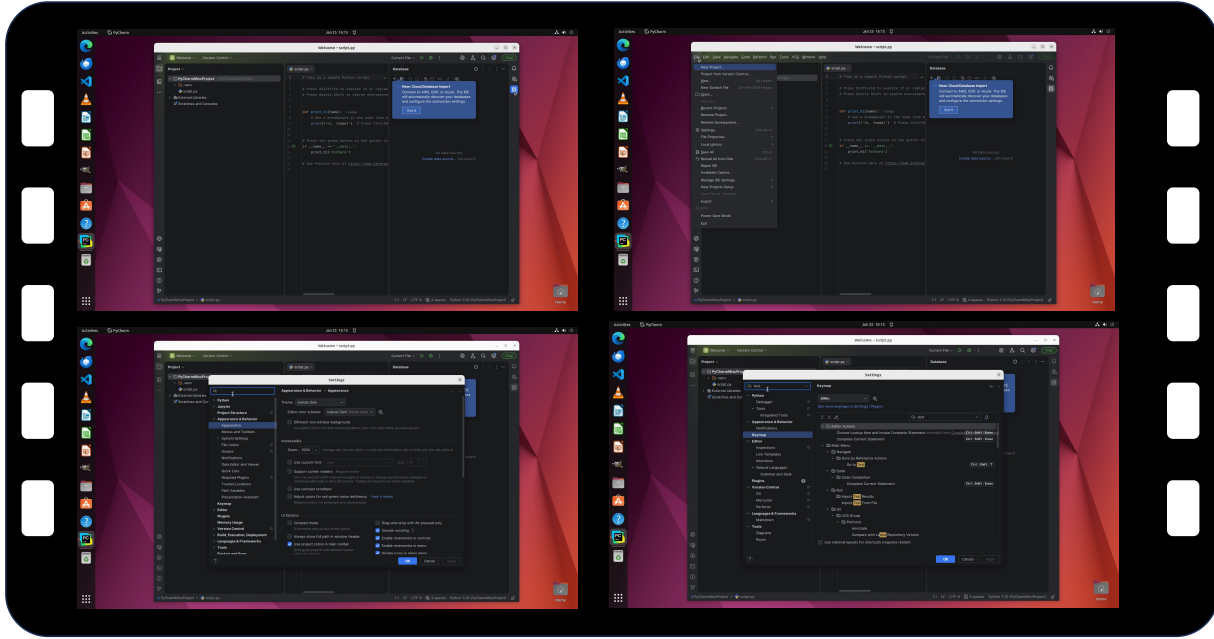


actual_url: <https://studio.blender.org/training/blender-2-8-fundamentals/knife-tool>
expected_url: https://docs.blender.org/manual/en/latest/movie_clip/tracking/clip/marker.html

Figure 20. An error case of execute before retrieval.

Error Type: Action Grounding Failure

Task Instruction: Configuring PyCharm for testing by creating a Test Sources Root and marking a directory to store test code.



Steps

- **Step_1:** In the Project tool window (Alt+01), create a new directory in which you will store your test code.
- **Step_2:** Right-click the new directory and select Mark Directory As | Test Sources Root.
- **Step_3:** Ensure the folder is marked with the Test Root icon, indicating it is configured as a Test Sources Root.

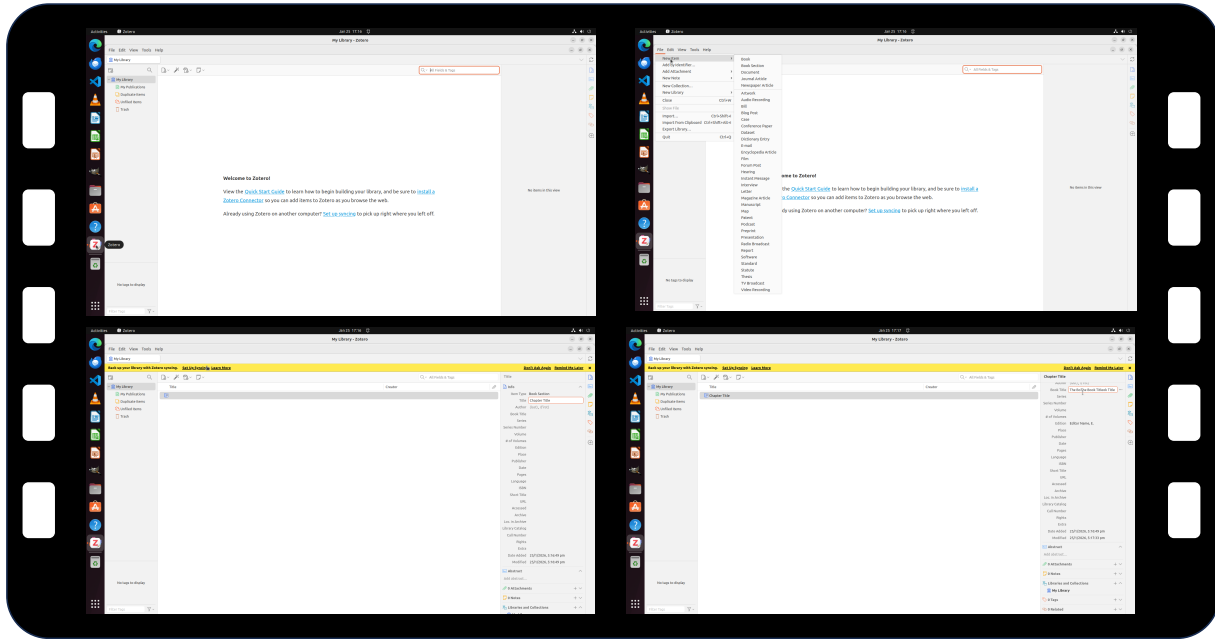
Explanation

- **Step 1 is completed:** the agent used the Project tool window context menu to create a new directory named 'test'.
- **Step 2 is completed:** the agent right-clicked the 'test' directory and selected Mark Directory As Test Sources Root.
- **Step 3 is not clearly completed** because the trajectory provides no verification that the folder shows the Test Root icon/visual marking after the action.

Figure 21. An error case of action grounding failure.

Error Type: Context Misidentification

Task Instruction: Configuring PyCharm for testing by creating a Test Sources Root and marking a directory to store test code.



Steps

- **Step_1:** Click the green new item button and select 'Book Section'.
- **Step_2:** Enter the chapter title in the Title field and the book title in the Book Title field.
- **Step_3:** To add an editor, click the '+' sign on the author line in the right column to create an additional author line.
- **Step_4:** Click the 'Author' label on the newly created line and change it to editor.

Explanation

- **Step 1 completed:** agent clicked the green New Item button and selected 'Book Section' (Steps 2\20133).
- **Step 2 completed:** agent entered a chapter title in Title ('The Quick Python Book') and a book title in Book Title ('Python Tricks').
- **Steps 3 & 4 not completed as specified:** the agent changed an existing Author role to Editor but did not click the '+' to add an additional creator line and then change that new line's role to Editor.

Figure 22. An error case of context misidentification.

D. Semantic Retrieval Evaluation Metric

To further strengthen the evaluation of the retrieval phase, we complement the original URL-based metrics with an additional semantic relevance metric. Specifically, we compute semantic similarity between the retrieved documents and the corresponding ground-truth references using embedding-based representations. As shown in Table 8, the semantic relevance metric exhibits a similarity score of 0.653 with the URL-based evaluation results, which indicates that the URL-based metrics are reasonably aligned with semantic relevance, supporting the validity of our original evaluation protocol.

Table 8. Performance comparison of various GUI agents with Sentence-BERT.

Models	Sentence-BERT
GELab-Zero-4B-preview	18.62
UI-TARS-1.5-7B	26.23
MAI-UI-8B	32.19
Aguvis-7B	27.27
GUI-Owl-7B	10.04
Qwen3-VL-8B	37.88

E. Additional Baselines

To address potential limitations in the diversity of baselines, we extend our experimental setup by incorporating both a larger-scale model and an additional agent framework. Specifically, we include the Qwen3-VL-32B as a higher-parameter baseline and integrate the AutoGen framework based on Qwen3-VL-7B as an alternative agent-based system. The results of these additional experiments are shown in Table 9. Overall, we observe that while larger-scale models demonstrate improved capabilities in certain aspects, the relative trends and conclusions reported in the main paper remain consistent. This suggests that our findings are not limited to a specific model scale or agent framework.

Table 9. Performance of larger-scale model and agentic framework.

Models	TUI	HPP	TCR
Qwen3-VL-32B	45.66	55.74	6.21
AutoGen (Qwen3-VL-8B)	41.21	53.63	5.13

F. Prompt Template

Table 10. Prompt Template for LLM as a Judge.

Prompt Template for LLM as a Judge.

You are an expert evaluator of GUI task execution. Based on the Ground Truth steps, evaluate how many of them the GUI Agent has completed.

Task description:

{task_description}

Ground Truth steps:

{ground_truth_steps}

GUI Agent trajectory:

{trajectory_text}

Guidelines:

- Count only steps the agent clearly completed or was very close to completing successfully.
- If the agent attempted but did not succeed, do NOT count it.
- If the agent performed extra detail beyond Ground Truth, count only the corresponding Ground Truth step(s).

Output format:

```
{
  "steps": <number of completed Ground Truth steps>,
  "Explanation": "<concise but specific explanation>"
}
```

Figure 23. Prompt for GUI Content Classification.

Prompt for Document Content Classification

You are an assistant that classifies content based on specific criteria. Your task is to evaluate whether a given piece of content serves as a tutorial specifically related to graphical user interfaces (GUI), such as web applications, desktop applications, or operating systems.

The content qualifies as a GUI-related tutorial if it meets the following conditions:

1. It includes a task description outlining what needs to be achieved.
2. It provides clear step-by-step instructions for interacting with a GUI, such as:
 - Step 1: Open the application.
 - Step 2: Navigate to the settings menu.

Given the URL and context, determine whether the content is a GUI-related tutorial. Output 1 if it is a GUI-related tutorial and 0 otherwise. Provide only the number as the output.

User Input:

- URL: {url}
- Context: {context}